

# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgavi-590018



A Project Phase -II (BIS786)

Report on

## “Alzheimer's Disease Stage Classification Using VGG16-based Deep Learning on Brain MRI Images”

*Submitted in partial fulfillment of the requirement for award of degree*

*of*

**BACHELOR OF ENGINEERING**

*in*

**INFORMATION SCIENCE AND ENGINEERING**

*by*

**RAMYA M**

**1EP22IS086**

**SHARAN KUMAR**

**1EP22IS099**

**SUNITHA SEERVI**

**1EP22IS111**

**VISHWA ARADHYA S M**

**1EP22IS121**

Under the guidance of

**Prof . Tejaswini B**

**Asst. Professor**

**Dept. of ISE, EPCET**



**EAST  
POINT**

**COLLEGE OF ENGINEERING &  
TECHNOLOGY**

**DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING**

**Jnana Prabha, Bidarahalli, Virgo Nagar Post, Bengaluru, Karnataka 560049**

**2025-26**

## **CERTIFICATE**

This is to certify that the project work phase II (BIS786) entitled **“Alzheimer’s Disease Stage Classification Using VGG16-based Deep Learning on Brain MRI Images”** is a bona-fide work carried out by **RAMYA M [1EP22IS086]**, **SHARAN KUMAR [1EP22IS099]**, **SUNITHA SEERVI [1EP22IS111]**, and **VISHWA ARADHYA S M [1EP22IS121]** in the partial fulfillment of the requirements of VII semester of **BACHELOR OF ENGINEERING** in INFORMATION SCIENCE AND ENGINEERING, VISVESVARAYA TECHNOLOGICAL UNIVERSITY, Belagavi, during the year 2025-26. It is certified that corrections/ suggestions indicated for internal assessment have been incorporated in this Project Phase II report deposited in the department library. The project report has been approved as it satisfies the academic requirements in respect of prescribed for the University.

**Prof. Tejaswini B**  
**Project Guide**  
**Department of ISE**

**Dr. Shanthi M**  
**HOD - ISE**  
**EPCET, Bangalore**

**Dr. Pradipkumar Dixit**  
**Principal**  
**EPCET, Bangalore**

**External Viva:**

**Name of the Examiners**

**Signature with date**

1.

2.

## ACKNOWLEDGEMENT

Any achievement, be it scholastic or otherwise does not depend solely on the individual efforts but on the guidance, encouragement and cooperation of intellectuals, elders and friends. I would like to take this opportunity to thank them all.

We would like to express our sincere thanks to the management of EPCET for providing necessary infrastructure and creating good environment.

We express our gratitude to **Dr. Pradipkumar Dixit**, Principal, EPCET who has always been a great source of inspiration.

We would like to express our sincere thanks to **Dr. Shanthi M**, Professor and Head of the Department of Information Science and Engineering, EPCET for her valuable suggestions and encouragement to do our best in this project work.

We would like to express our gratitude towards our guide **Prof. Tejaswini B**, Assistant Professor , Department of ISE and the project coordinator **Prof. Tejaswini B** Assistant Professor and **Dr. Udayabalan B** Professor , Department of ISE for their valuable guidance and constant supervision in completing the project work successfully.

We would like to extend our thanks to all the faculty members of ISE department for their valuable inputs as reviewers during the course of the project work.

Finally, we would like to thank our parents and friends for their support and encouragement in successful completion of the project work.

|                           |                     |
|---------------------------|---------------------|
| <b>RAMYA M</b>            | <b>[1EP22IS086]</b> |
| <b>SHARAN KUMAR</b>       | <b>[1EP22IS099]</b> |
| <b>SUNITHA SEERVI</b>     | <b>[1EP22IS111]</b> |
| <b>VISHWA ARADHYA S M</b> | <b>[1EP22IS121]</b> |

# ABSTRACT

Alzheimer's disease is a progressive neurodegenerative disorder characterized by memory loss, cognitive decline, and behavioural changes. Early detection is crucial for effective treatment planning and patient care. This project presents an automated Alzheimer's disease stage classification system using deep learning, specifically the VGG16 Convolutional Neural Network (CNN) architecture. The system is designed to classify MRI brain images into four categories: Non-Demented, Very Mild Demented, Mild Demented, and Moderate Demented.

The proposed approach includes a complete workflow consisting of dataset collection, image preprocessing, normalization, resizing, and augmentation to enhance dataset diversity. Transfer learning is applied using the VGG16 pre-trained model, and customized fully connected layers are added for multiclass classification. The model is trained, fine-tuned, and evaluated using performance metrics such as accuracy, precision, recall, and F1-score. The system achieves high classification accuracy and demonstrates reliable performance in distinguishing between different dementia stages.

A Flask-based web application is implemented to provide a user-friendly interface where users can upload MRI images and receive real-time predictions along with confidence scores. The system is lightweight, scalable, and suitable for integration into clinical support environments.

This project demonstrates that deep learning models, when combined with proper preprocessing and transfer learning, can significantly improve the accuracy and efficiency of Alzheimer's stage detection, offering a valuable tool for healthcare professionals and researchers.

|                    | <b>CONTENTS</b>                           |                 |
|--------------------|-------------------------------------------|-----------------|
| <b>Chapter No.</b> | <b>Description</b>                        | <b>Page No.</b> |
| <b>1</b>           | <b>Introduction</b>                       | 1-6             |
|                    | 1.1 Background                            | 2               |
|                    | 1.2 Problem Statement                     | 3               |
|                    | 1.3 Existing System                       | 3               |
|                    | 1.4 Proposed System                       | 4               |
|                    | 1.5 Objective of Project                  | 5               |
| <b>2</b>           | <b>Literature Survey</b>                  | 7-9             |
| <b>3</b>           | <b>Requirements Specification</b>         | 10-12           |
|                    | 3.1 Requirement Specifications            | 10              |
|                    | 3.2 System Requirements                   | 11              |
|                    | 3.3 Software Requirements                 | 11              |
|                    | 3.4 Hardware Requirements                 | 12              |
| <b>4</b>           | <b>System Analysis and System Design</b>  | 13-19           |
|                    | 4.1 System Analysis                       | 13              |
|                    | 4.2 System Design                         | 15              |
| <b>5</b>           | <b>System Implementation</b>              | 20-45           |
|                    | 5.1 Dataset Collection and Preprocessing  | 20              |
|                    | 5.2 Model Selection and Transfer Learning | 21              |
|                    | 5.3 Model Training and Evaluation         | 22              |
|                    | 5.4 Deployment and Real-Time Prediction   | 22              |
|                    | 5.5 APPENDIX – CODE SNIPPET               | 23-45           |
| <b>6</b>           | <b>Tools and Technologies</b>             | 46-66           |
|                    | 6.1 About Python                          | 46              |

|          |                                    |       |
|----------|------------------------------------|-------|
|          | 6.2 About Deep Learning            | 51    |
|          | 6.3 Flask                          | 53    |
|          | 6.4 Html,Css and VsCode            | 57    |
|          | 6.5 VGG16 Algorithm                | 62    |
|          | 6.6 Test Plan                      | 64    |
|          | <b>Verification and Validation</b> | 67-69 |
| <b>7</b> | 7.1 Verification                   | 67    |
|          | 7.2 Validation                     | 68    |
|          | 7.3 Validation Test Result         | 69    |
|          | <b>Result and Conclusion</b>       | 70-75 |
| <b>8</b> | 8.1 Result Analysis                | 70    |
|          | 8.2 Result Snapshots               | 71-74 |
|          | 8.3 Conclusion                     | 74    |
|          | 8.4 Future Enhancement             | 75    |
|          | <b>Reference</b>                   | 76    |

## LIST OF FIGURES

| <b>Fig. no.</b> | <b>Description</b>          | <b>Page no.</b> |
|-----------------|-----------------------------|-----------------|
| 4.1             | System Architecture         | 13              |
| 4.2             | Flow Chart Diagram          | 15              |
| 4.3             | Use Case Diagram            | 17              |
| 6.1             | VGG16 Visual Geometry Group | 62              |
| 6.2             | Test Plan                   | 65              |
| 8.1             | Home Page                   | 71              |
| 8.2             | About Page                  | 72              |
| 8.3             | Performance Page            | 72              |
| 8.4             | Predict Page                | 73              |
| 8.5             | Uploading MRI Images        | 73              |
| 8.6             | Result Page                 | 74              |

## LIST OF TABLES

| <b>Table no.</b> | <b>Description</b>             | <b>Page no.</b> |
|------------------|--------------------------------|-----------------|
| 2.1              | Summary of Reviewed Literature | 9               |
| 3.1              | System Requirement             | 11              |
| 3.2              | Software Requirement           | 12              |
| 3.3              | Hardware Requirement           | 12              |
| 7.1              | Module Verification            | 68              |
| 7.2              | Functional Validation          | 69              |



## CHAPTER 1

### INTRODUCTION

Alzheimer's disease (AD) is a chronic neurodegenerative disorder that leads to a gradual decline in cognitive function, affecting memory, thinking, and behavior. Early detection of Alzheimer's is crucial as it allows for timely interventions and better patient management. Traditionally, the diagnosis of Alzheimer's disease relies on clinical evaluations, cognitive tests, and medical imaging techniques, particularly MRI scans, which reveal patterns of brain atrophy associated with different stages of the disease.

Recent advancements in machine learning, especially deep learning, have revolutionized medical image analysis by enabling the automatic classification of various diseases with high accuracy. Convolutional Neural Networks (CNNs) have proven to be highly effective in image classification tasks due to their ability to automatically extract features from images without requiring manual feature engineering.

In this project, we aim to classify Alzheimer's disease stages using the VGG16 deep learning model, a CNN architecture known for its simplicity and effectiveness in image classification. The project focuses on four stages of the disease: Non-Demented, Very Mild Demented, Mild Demented, and Moderate Demented. By training the VGG16 model on a dataset of MRI images, the system can automatically classify these stages, offering a potential tool for early detection and diagnosis of Alzheimer's.

This project demonstrates the power of transfer learning, where a pre-trained VGG16 model, originally designed for general image classification, is fine-tuned to identify Alzheimer's stages. The ultimate goal is to build a robust and accurate system that can assist healthcare professionals in diagnosing Alzheimer's disease, contributing to better treatment planning and patient outcomes.

## 1.1 Background

Alzheimer's disease, first identified by Dr. Alois Alzheimer in 1906, is the most common form of dementia, accounting for nearly 60–80% of all dementia cases worldwide. It is characterized by the progressive degeneration of brain cells, leading to memory loss, confusion, and difficulties in performing everyday activities. The exact cause of Alzheimer's remains unknown, but it is associated with abnormal protein deposits in the brain—amyloid plaques and tau tangles—that disrupt neuronal communication.

Medical imaging, especially Magnetic Resonance Imaging (MRI), plays a significant role in the diagnosis and monitoring of Alzheimer's disease. MRI scans provide detailed structural images of the brain, allowing clinicians to observe atrophy patterns, particularly in regions such as the hippocampus and cerebral cortex. However, manual interpretation of these images is time-consuming, subjective, and prone to human error.

With the rapid growth of Artificial Intelligence (AI) and Deep Learning (DL), automated diagnostic systems have emerged as a powerful alternative to traditional diagnostic methods. Convolutional Neural Networks (CNNs), in particular, have shown exceptional performance in medical image classification due to their hierarchical feature extraction capabilities. Among the various CNN architectures, VGG16 has become a widely adopted model for medical image analysis because of its simple yet deep architecture that effectively captures spatial hierarchies in images.

In this project, VGG16 is employed through transfer learning, where a pre-trained model—originally trained on large-scale image datasets like ImageNet—is fine-tuned using MRI brain images specific to Alzheimer's disease. This approach significantly reduces training time and improves model accuracy, even with a relatively small medical dataset. The model is trained to distinguish between four stages of Alzheimer's—Non-Demented, Very Mild Demented, Mild Demented, and Moderate Demented—providing an automated, accurate, and reliable classification framework.

This background establishes the foundation for leveraging deep learning in medical diagnostics and highlights the significance of early detection in managing Alzheimer's disease.

## 1.2 Problem Statement

Alzheimer's disease is a progressive neurodegenerative disorder that affects millions of individuals worldwide, leading to memory loss, cognitive decline, and ultimately, a loss of independence. Early detection and accurate staging of the disease are critical for effective treatment, care, and slowing the progression of symptoms. However, current diagnostic methods can be time-consuming, expensive, and require the expertise of specialists, making early diagnosis a challenge.

The primary goal of this project is to develop an automated system for classifying the stages of Alzheimer's disease using deep learning techniques, specifically the VGG16 convolutional neural network (CNN) architecture. By leveraging MRI brain images, the system aims to differentiate between four categories: Non-Demented, Very Mild Demented, Mild Demented, and Moderate Demented, aiding in the early detection and staging of Alzheimer's disease.

The key problem this project addresses is the need for an efficient, accurate, and accessible method of classifying the stages of Alzheimer's disease from brain MRI images, supporting medical professionals in making informed decisions and facilitating timely interventions.

## 1.3 Existing System

The traditional systems for diagnosing Alzheimer's disease primarily rely on manual examination of brain MRI or CT scans by neurologists and radiologists. These methods involve visual inspection of brain regions to identify abnormalities such as cortical shrinkage or hippocampal atrophy. The diagnosis process is time-consuming, labour-intensive, and depends heavily on the expertise of medical professionals, which can lead to subjective interpretations and inconsistent results.

Earlier computer-based systems often used **basic image processing** and **traditional machine learning** techniques such as thresholding, edge detection, Support Vector Machines (SVM), k-Nearest Neighbour (k-NN), and Random Forests. These approaches required **manual feature extraction** from MRI images, demanding extensive domain knowledge to identify relevant features. However, such handcrafted features may fail to capture subtle and complex structural variations in the

brain associated with early Alzheimer's stages.

Moreover, these conventional systems faced **several drawbacks**:

- **Low Accuracy and Generalization:** Traditional methods struggled to achieve high accuracy due to limited feature extraction capabilities and smaller datasets.
- **Manual Feature Engineering:** Required human intervention for selecting and extracting features, increasing the risk of human error.
- **Poor Scalability:** Unable to handle large and complex MRI datasets efficiently.
- **Limited Stage Classification:** Most existing systems could only distinguish between healthy and demented patients, lacking the ability to classify intermediate stages (e.g., very mild or mild dementia).
- **Time-Consuming Diagnosis:** Manual assessment of MRI scans delayed the diagnosis process, impacting early intervention and treatment planning.

Due to these limitations, there is a growing need for **automated, accurate, and scalable deep learning-based systems** capable of analyzing MRI images directly and classifying different stages of Alzheimer's disease without manual intervention.

## 1.4 Proposed System

The proposed system introduces an automated **deep learning-based approach** for the classification of Alzheimer's disease stages using **Magnetic Resonance Imaging (MRI)** brain scans. The system leverages the **VGG16 Convolutional Neural Network (CNN)** architecture, which is well-known for its superior image classification performance and feature extraction capabilities. By applying **transfer learning**, the model adapts the pre-trained VGG16 network (originally trained on ImageNet) to efficiently classify MRI images into four distinct categories — **Non-Demented, Very Mild Demented, Mild Demented, and Moderate Demented**.

Unlike traditional systems that depend on manual feature extraction, this model automatically learns intricate and high-level features from raw MRI data, significantly improving accuracy and reducing human dependency. The deep learning model processes large datasets to identify subtle structural changes in brain anatomy that are often difficult to detect manually.

The system's workflow includes **data preprocessing**, **model training**, **validation**, and **real-time prediction** through a user-friendly interface. Image preprocessing techniques such as resizing, normalization, and data augmentation ensure that the model receives optimized input for efficient learning. After training, the model is integrated into a **Flask-based web application**, allowing users to upload MRI images and receive instant predictions along with probability scores for each Alzheimer's stage.

#### **Key Features of the Proposed System:**

- **Automated Classification:** Eliminates manual intervention by automatically identifying Alzheimer's stages from MRI scans.
- **High Accuracy:** Utilizes deep learning (VGG16) with transfer learning to achieve improved precision and reliability in classification.
- **Efficient and Scalable:** Capable of handling large datasets and real-time processing for medical applications.
- **User-Friendly Interface:** Enables easy MRI image uploads and displays clear diagnostic predictions.
- **Performance Evaluation:** Provides detailed metrics such as accuracy, precision, recall, and F1-score to validate model performance.

#### **Advantages over Existing Systems:**

- Reduces dependency on expert interpretation.
- Improves classification accuracy and consistency.
- Enables early detection and diagnosis of Alzheimer's stages.
- Offers faster and more accessible results for healthcare professionals.

The proposed system thus combines **deep learning efficiency** with **medical imaging accuracy**, offering a reliable, scalable, and automated diagnostic tool to assist clinicians in the early detection and staging of Alzheimer's disease.

## **1.5 Objectives of the Project**

The main objective of this project is to develop an **automated deep learning-based system** that can accurately classify different stages of **Alzheimer's disease** using **brain MRI images**. The system aims to assist medical professionals in early diagnosis, improve decision-making, and enhance patient

care through reliable and efficient image analysis.

### **Specific Objectives:**

- 1. To develop a deep learning model using the VGG16 architecture**
  - Design and implement a Convolutional Neural Network (CNN)-based model capable of classifying MRI brain images into four Alzheimer's stages: Non-Demented, Very Mild Demented, Mild Demented, and Moderate Demented.
- 2. To apply transfer learning for improved feature extraction**
  - Fine-tune the pre-trained VGG16 model (trained on ImageNet) to effectively capture disease-specific features from MRI data, enhancing accuracy with limited medical datasets.
- 3. To preprocess and augment MRI image data**
  - Perform image resizing, normalization, and data augmentation to improve model generalization and prevent overfitting.
- 4. To evaluate model performance using standard metrics**
  - Assess accuracy, precision, recall, F1-score, and confusion matrix to measure the system's classification performance.
- 5. To develop a user-friendly web interface for predictions**
  - Create a Flask-based application allowing users to upload MRI images and obtain real-time classification results with corresponding probability scores.
- 6. To assist in early detection and decision-making**
  - Provide healthcare professionals with an intelligent and automated diagnostic tool for timely identification and staging of Alzheimer's disease.
- 7. To ensure scalability and future integration**
  - Design the system to accommodate larger datasets, future enhancements, and potential integration with hospital management systems.

## CHAPTER 2

### LITERATURE SURVEY

**Paper [1] — *Alzheimer's Disease Classification Using Convolutional Neural Networks* (Liu, S.; Liu, S.; Cai, W.; Pujol, S.; Kikinis, R.; Feng, D.)**

- **Abstract:** The authors propose a CNN-based method for Alzheimer's classification using **3D MRI** images, showing that deep learning can automatically detect patterns in brain scans without manual feature engineering and that CNNs outperform traditional ML approaches.
- **Key work done:**
  1. Designed and trained a 3D CNN to exploit volumetric MRI data and spatial context.
  2. Demonstrated higher accuracy compared to traditional ML pipelines that rely on handcrafted features (reported accuracy ~88.9%).
  3. Emphasized automatic feature extraction from raw MRI volumes, reducing the need for domain-specific feature engineering.
- **Limitations:**
  1. 3D CNNs are computationally expensive—require more memory and GPU resources, making training and deployment heavier.
  2. Performance can degrade with small/imbalanced datasets; 3D models demand larger annotated volumes to generalize well.
  3. Less interpretable than some classical methods; identifying which anatomical features drive decisions remains challenging.

**Paper [2] — *Deep Learning-Based Alzheimer's Disease Diagnosis Using Structural MRI* (Suk, H.I.; Lee, S.W.; Shen, D.)**

- **Abstract:** This work applies deep learning — notably a VGG-like CNN — to structural MRI for multiclass Alzheimer's staging (Non-Demented, Very Mild, Mild, Moderate), focusing on architecture optimization for multiclass classification and reporting competitive accuracy (~87.1%).
- **Key work done:**
  1. Adapted a deep CNN (VGG-style) and fine-tuned it for MRI-derived features to classify multiple stages rather than a binary healthy/diseased split.
  2. Applied preprocessing and augmentation tailored to structural MRI to improve generalization on limited medical datasets.
  3. Evaluated using standard metrics (accuracy, confusion matrix, precision/recall) to analyse stage-wise performance.
- **Limitations:**
  1. VGG-style networks (2D pre-trained on ImageNet) might not fully exploit 3D volumetric information from MRI unless slices/volumes are carefully handled.

2. Transfer learning from natural images (ImageNet) to medical MRI has domain-shift issues — low-level features differ between domains.
3. Performance at fine-grained staging (very mild vs mild) can be limited when inter-stage differences are subtle and dataset labels noisy.

**Paper [3] — *3D CNN for Classification of Alzheimer's Disease Stages Using Structural MR Images* (Payan, A.; Montana, G.)**

- **Abstract:** The authors propose a 3D CNN that leverages volumetric MRI to capture spatial relationships across slices, reporting improved accuracy ( $\approx 89.5\%$ ) and showing that 3D models can better exploit MRI's full information content.
- **Key work done:**
  1. Built and trained a 3D convolutional architecture taking whole-volume MRIs as input, preserving inter-slice context.
  2. Compared 3D vs 2D approaches and reported gains in capturing structural atrophy patterns relevant to Alzheimer's.
  3. Showed the feasibility of end-to-end volumetric learning for staging (AD / MCI / healthy).
- **Limitations:**
  1. Heavy compute and memory footprint — 3D networks need more GPUs and longer training times.
  2. Requires consistent, pre-aligned volumetric inputs (registration), increasing preprocessing complexity.
  3. Risk of overfitting when dataset size is limited — 3D parameters scale quickly with model depth.

**Paper [4] — *Automatic Classification of Alzheimer's Disease and Mild Cognitive Impairment Using 2D Brain MRI Scans* (Basaia, S.; Agosta, F.; Wagner, L.)**

- **Abstract (short):** This study uses 2D MRI slices and transfer learning with VGG16 for Alzheimer's and MCI classification, demonstrating that a pre-trained VGG16 fine-tuned on MRI slices outperforms some custom CNNs (reported accuracy  $\sim 88.4\%$ ).
- **Key work done:**
  1. Employed transfer learning on VGG16 applied to selected informative 2D slices from MR volumes, leveraging ImageNet-pretrained weights to speed training.
  2. Compared VGG16-based transfer learning with bespoke CNNs, showing benefits in accuracy and faster convergence.
  3. Used slice selection and aggregation strategies to produce subject-level decisions from slice-level predictions.



- **Limitations:**

1. 2D slice-based methods can lose 3D contextual information; performance depends on which slices are chosen.
2. Transfer learning from natural images has domain mismatch; some mid/high-level MRI features may still be suboptimal.
3. Aggregation strategy (how slice predictions combine into a final subject prediction) can introduce variance and reduce interpretability.

**Short synthesis (what these papers teach us)**

- 3D CNNs can capture volumetric context better and sometimes achieve higher accuracy (but at higher computational cost).
- 2D slice-based transfer learning (e.g., VGG16) is effective and computationally cheaper, yet may lose volumetric detail; it benefits from careful slice selection and aggregation.
- Transfer learning from ImageNet is a practical approach but carries domain-shift limitations; dedicated medical pretraining or careful fine-tuning improves results.
- Dataset size, preprocessing (registration/normalization), and class imbalance strongly affect reported performance across studies.

| Reference                                                                                                                    | Major Findings                                                                                                                  | Methods                                                                                                             | Limitations                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| [1] Liu et al. (“Alzheimer’s Disease Classification Using Convolutional Neural Networks”)                                    | 3D volumetric CNN models effectively learn spatial patterns from MRI scans and outperform traditional machine learning methods. | Implemented a 3D CNN on whole MRI volumes for end-to-end feature learning and classification of Alzheimer’s stages. | Requires high computational resources and large datasets; less interpretable and more prone to overfitting.           |
| [2] Suk et al. (“Deep Learning-Based Alzheimer’s Disease Diagnosis Using Structural MRI”)                                    | Demonstrated that VGG-style CNN architectures can be adapted for multiclass Alzheimer’s staging with good accuracy (~87.1%).    | Used 2D VGG-based CNN with transfer learning and data augmentation for multiclass classification of MRI images.     | Domain shift from natural to medical images; limited use of 3D context unless carefully handled.                      |
| [3] Payan & Montana (“3D CNN for Classification of Alzheimer’s Disease Stages Using Structural MR Images”)                   | Showed that 3D CNNs capture anatomical atrophy patterns more accurately than 2D methods (accuracy ≈ 89.5%).                     | Trained a 3D convolutional architecture using registered MR volumes to leverage volumetric information.             | Heavy computational cost; requires precise pre-processing and large training data to avoid overfitting.               |
| [4] Basaia et al. (“Automatic Classification of Alzheimer’s Disease and Mild Cognitive Impairment Using 2D Brain MRI Scans”) | Applied transfer learning with VGG16 on 2D MRI slices, achieving high accuracy (~88.4%) and efficient training.                 | Used 2D slice-based transfer learning (VGG16) with slice selection and prediction aggregation.                      | Loss of 3D spatial information; results depend on slice selection and aggregation strategy; domain mismatch possible. |

Table 2.1 : Summary of Reviewed Literature

## CHAPTER 3

# REQUIREMENT SPECIFICATIONS

### 3.1 Requirement Specification

The proposed system aims to classify different stages of Alzheimer's disease using MRI brain images through a deep learning approach. For the system to operate efficiently, both software and hardware components must be compatible to support data preprocessing, model training, and web-based deployment.

The system requirements are divided into **functional** and **non-functional** specifications:

#### Functional Requirements

- The system shall allow users to upload MRI images through a web interface.
- The system shall preprocess uploaded images (resize, normalize, and format).
- The system shall classify the uploaded MRI into one of the four categories: *Non-Demented*, *Very Mild Demented*, *Mild Demented*, or *Moderate Demented*.
- The system shall display the classification result and corresponding probability.
- The system shall log any errors or failures for debugging and performance improvement.

#### Non-Functional Requirements

- **Performance:** The model must classify an image in under 5 seconds.
- **Usability:** Interface should be simple and intuitive for non-technical users.
- **Scalability:** The architecture should allow training with larger datasets or newer models.
- **Accuracy:** The classification model should maintain accuracy  $\geq 85\%$ .
- **Security:** Uploaded images must be handled securely, and data should not be stored without consent.
- **Compatibility:** The system should run smoothly on all modern browsers and Windows platforms.

### 3.2 System Requirement

To implement the Alzheimer's Disease Classification System, the following setup is required:

| Category                | Specification                                                       |
|-------------------------|---------------------------------------------------------------------|
| • Operating System      | Windows 10 or higher                                                |
| • Programming Language  | Python                                                              |
| • Development IDE       | Visual Studio Code / Python IDLE                                    |
| • Framework             | Flask (for backend web deployment)                                  |
| • Frontend Technologies | HTML, CSS, JavaScript                                               |
| • Libraries Used        | TensorFlow / Keras, NumPy, Pandas, OpenCV, Matplotlib, Scikit-learn |
| • Dataset               | Alzheimer's MRI Dataset (Kaggle / ADNI)                             |

Table 3.1 : System Requirement

The system setup ensures smooth execution of both the machine learning model training and the web-based prediction interface.

### 3.3 Software Requirements

The software components necessary for development, training, and deployment of the model are as follows:

| Software                  | Description                                     |
|---------------------------|-------------------------------------------------|
| • Operating System        | Windows 10 or above (64-bit)                    |
| • Python Version          | Python 3.7 or later                             |
| • Development Environment | Visual Studio Code or Jupyter Notebook          |
| • Framework               | Flask (for web application deployment)          |
| • Deep Learning Libraries | TensorFlow, Keras                               |
| • Supporting Libraries    | NumPy, Pandas, OpenCV, Scikit-learn, Matplotlib |

|                    |                                |
|--------------------|--------------------------------|
| • Web Technologies | HTML, CSS, JavaScript          |
| • Package Manager  | pip (for library installation) |

Table 3.2 : Software Requirement

### 3.4 Hardware Requirements

| Component           | Minimum Requirement                            | Recommended Specification              |
|---------------------|------------------------------------------------|----------------------------------------|
| Processor           | Intel Core i3                                  | Intel Core i5 or higher                |
| RAM                 | 4 GB                                           | 8 GB or more (for training efficiency) |
| Hard Disk / SSD     | 20 GB free space                               | 50 GB free space                       |
| GPU (optional)      | Not mandatory                                  | NVIDIA GPU (for faster model training) |
| Display             | 1080p Resolution                               | Full HD or higher                      |
| Internet Connection | Required for dataset download and dependencies | Stable broadband recommended           |

Table 3.3 : Hardware Requirements

## CHAPTER 4

# SYSTEM ANALYSIS AND SYSTEM DESIGN

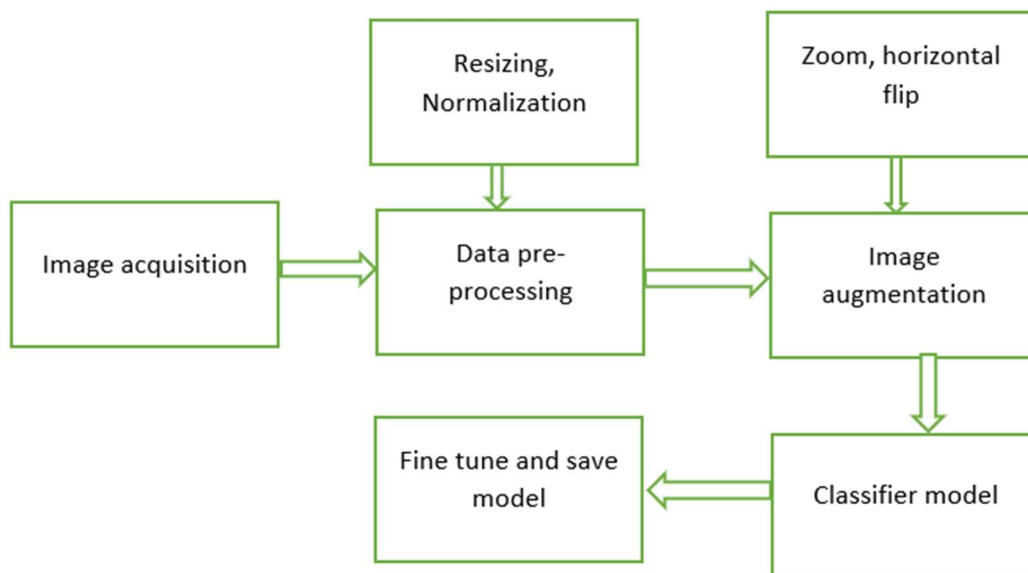
### 4.1 System Analysis

System analysis is the process of examining the requirements and functionalities of the proposed system to understand how it operates and how the individual components interact. In this project, the proposed system aims to classify Alzheimer's disease stages using deep learning (VGG16) on MRI brain images. The system automates the diagnostic process by analysing brain scans and predicting the stage of Alzheimer's disease with high accuracy.

The proposed system is divided into multiple functional modules, each performing a specific task to ensure accurate classification and efficient data processing. These modules work together to form a complete deep learning pipeline, starting from data acquisition to final prediction.

#### 4.1.1 System Architecture

The system architecture of the proposed Alzheimer's Disease Classification model is illustrated in Figure 4.1. It shows the major components involved in data flow and processing, starting from dataset collection to final output generation.



**Figure 4.1: System Architecture**

**Explanation:**

As shown in **Figure 4.1**, the architecture consists of several sequential stages:

**1. Image Acquisition:**

MRI brain images are collected from publicly available datasets such as *Kaggle* or *ADNI*. These images serve as the input to the system.

**2. Data Pre-processing:**

The collected images undergo preprocessing operations such as **resizing**, **normalization**, and **augmentation** (rotation, flipping, and zooming). This ensures that all images conform to a uniform format suitable for deep learning.

**3. Feature Extraction using VGG16:**

The **VGG16 model**, pre-trained on the ImageNet dataset, is used as a feature extractor. Transfer learning is applied by freezing the initial layers and fine-tuning the final layers to learn Alzheimer's-specific features.

**4. Model Training and Tuning:**

The processed data is divided into training and validation sets. The model is trained using **categorical cross-entropy loss** and **Adam optimizer**. Hyperparameter tuning is performed to achieve optimal performance and avoid overfitting.

**5. Validation and Testing:**

Once trained, the model is validated using unseen MRI data. The accuracy, precision, recall, and F1-score are computed to evaluate its performance.

**6. Classification and Output:**

The final trained model classifies MRI images into one of four stages: **Non-Demented**, **Very Mild Demented**, **Mild Demented**, and **Moderate Demented**. The predicted result and probability scores are displayed to the user.

**7. Deployment Interface:**

A **Flask-based web application** allows users to upload MRI scans. The backend processes the image using the trained model and displays the classification result in real-time.

This architecture ensures that the system performs accurate classification with an automated, end-to-end deep learning workflow.

## 4.2 System Design

System design defines the overall structure of the proposed system, including data flow, logical components, and user interaction. It represents how the system components are interconnected and how the data moves between different stages.

### 4.2.1 Flow Chart

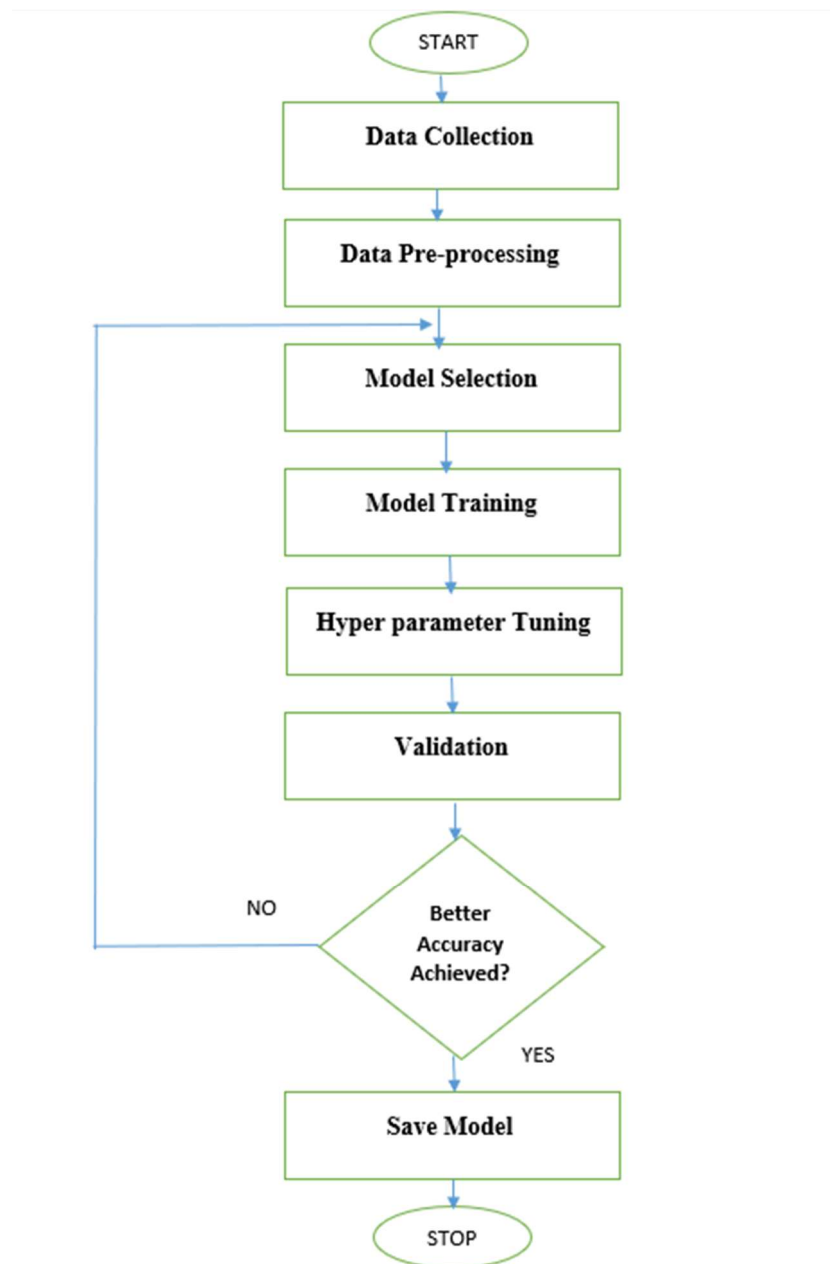


Figure 4.2: Flow Chart Diagram

The working flow of the proposed system is represented in Figure 4.2. It illustrates the step-by-step sequence of operations starting from data collection to final model deployment.

Explanation:

As shown in Figure 4.2, the flow starts with data collection, followed by pre-processing of MRI images. The next step is model selection and training, where the VGG16 architecture is used to classify the images.

After training, hyperparameter tuning is carried out to improve accuracy. If better accuracy is achieved, the model is saved; otherwise, training continues with new configurations. Once the model achieves the desired performance, it is validated and deployed for real-time predictions.

### **Workflow Description:**

The proposed system follows a structured pipeline to classify Alzheimer's Disease stages using a VGG16-based deep learning model. This process ensures that MRI images are properly prepared, the model is fine-tuned and validated, and only the most accurate model is saved for deployment. Each step in the workflow contributes to achieving reliable and consistent classification performance.

### **Step Descriptions:**

#### **1. Data Collection:**

MRI brain images are gathered from publicly available datasets or hospital archives to create a diverse training dataset.

#### **2. Data-Pre-processing:**

Images are resized (e.g., to 224×224 pixels) and normalized to a 0–1 range to ensure consistency and optimize model training.

#### **3. Model**

The VGG16 Convolutional Neural Network (CNN) is chosen for its proven effectiveness in image classification tasks. The model is adapted using transfer learning.

#### **4. Model-Training:**

The pre-processed images are fed into the model, and training is performed to learn patterns and features relevant to Alzheimer's Disease stages.

#### **5. Hyperparameter-Tuning:**

Model parameters such as learning rate, batch size, and epochs are adjusted to improve



performance and avoid overfitting.

**6. Validation:**

The model's performance is evaluated using a validation set. Metrics like accuracy, precision, and recall help determine its effectiveness.

**7. Better-Accuracy-Achieved?**

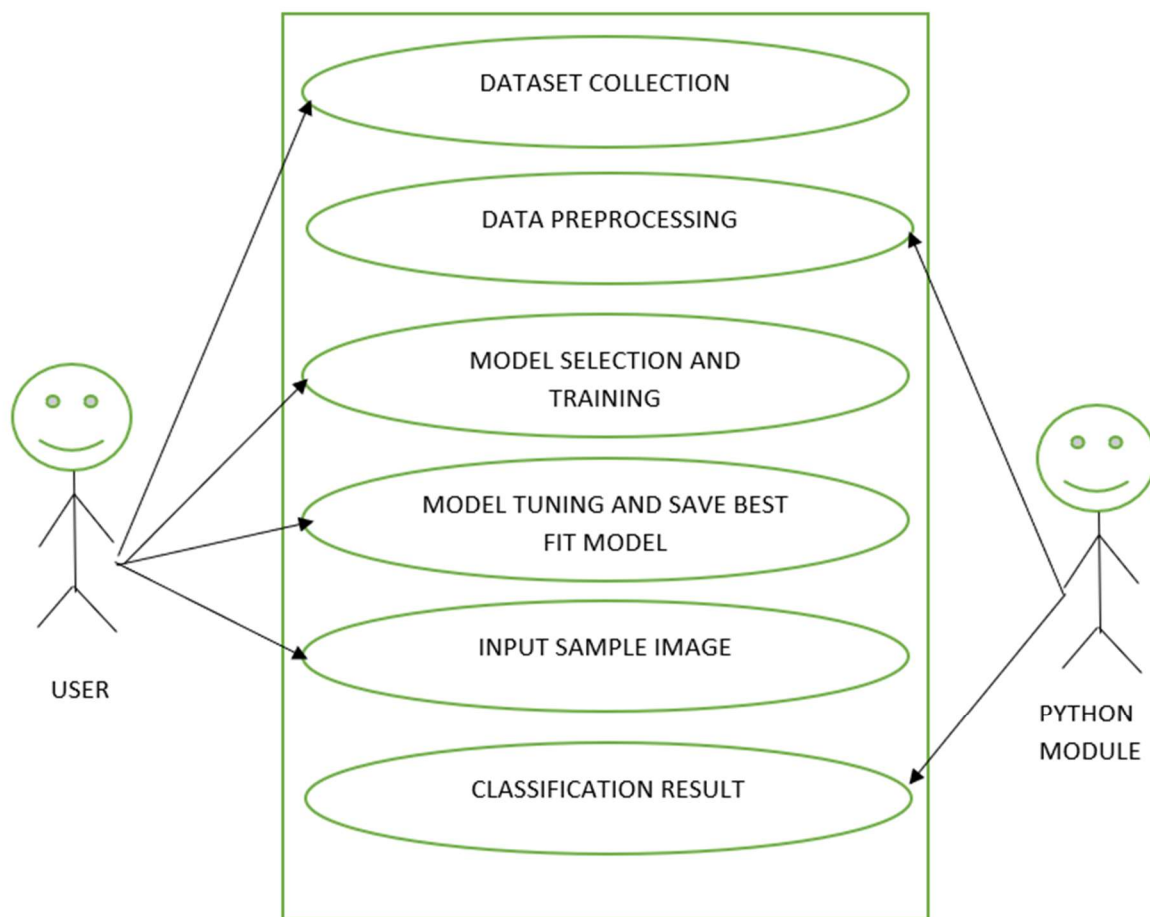
If the model's performance is not satisfactory, further hyperparameter tuning or adjustments are made. If accuracy is sufficient, the model proceeds to the next step.

**8. Save-Model:**

The final model with the best performance is saved for deployment in clinical or resea

### 4.2.2 Use Case Diagram

The use case diagram in Figure 4.3 illustrates the interaction between the user and the system.



**Figure 4.3: Use Case Diagram**

**Explanation:**

As shown in Figure 4.3, the primary actor in the system is the User (which can be a doctor, researcher, or general user).

The main use cases include:

- Uploading an MRI brain image
- Processing and classifying the image using the trained VGG16 model
- Viewing the predicted Alzheimer's stage
- Updating or saving the trained model (for admin/developer use)

The system internally handles data preprocessing, model tuning, and result generation before providing the output back to the user.

**Use Case Description:**

This use case diagram illustrates the interaction between the user and the Python-based module for Alzheimer's Disease Stage Classification. The diagram outlines the essential steps from dataset collection to the classification result, demonstrating how users can input MRI images and receive automated stage predictions through a deep learning model.

**Step Explanations:****1. Dataset Collection:**

Users gather MRI brain images from publicly available sources or medical archives to build a dataset for training and testing the model.

**2. Data-Preprocessing:**

Images are resized, normalized, and standardized to prepare them for model training. Preprocessing ensures that the data is consistent and suitable for deep learning workflows.

**3. Model Selection and Training:**

The user or developer selects the VGG16 CNN architecture, leveraging transfer learning, and trains it using the pre-processed dataset. This step allows the model to learn features relevant to Alzheimer's Disease stages.

**4. Model Tuning and Save Best Fit Model:**

Hyperparameters such as learning rate, batch size, and number of epochs are fine-tuned to achieve optimal performance. Once satisfactory accuracy is achieved, the trained model is saved for later use.

**5. Input Sample Image:**

The user uploads an MRI image that they want to classify. This step involves providing a new input to the trained model.

**6. Classification Result:**

The Python module processes the uploaded image using the trained VGG16 model and returns the predicted Alzheimer's Disease stage. This result helps medical professionals in early diagnosis and treatment planning.

## CHAPTER 5

### SYSTEM IMPLEMENTATION

This project focuses on classifying Alzheimer's disease stages using **VGG16**, a convolutional neural network (CNN), through **transfer learning**. The implementation involves multiple stages, including **data preprocessing, model selection, training, evaluation, and deployment**. Below is a step-by-step theoretical explanation of the implementation.

#### 5.1 Dataset Collection and Preprocessing

##### 5.1.1 Dataset Acquisition

- The dataset consists of **MRI brain scan images** categorized into four classes:
  1. **Non-Demented** (Healthy)
  2. **Very Mild Demented**
  3. **Mild Demented**
  4. **Moderate Demented**
- The images are sourced from publicly available datasets like **Kaggle's Alzheimer's MRI Dataset** or **ADNI (Alzheimer's Disease Neuroimaging Initiative)**.

##### 5.1.2 Data Preprocessing

To ensure optimal model performance, preprocessing steps include:

- **Resizing Images:** Standardize all images to **224×224 pixels**, the input size for VGG16.
- **Normalization:** Scale pixel values to a range of **0-1** (dividing by 255) for faster convergence.
- **Data Augmentation:** Apply transformations (rotation, flipping, zooming, etc.) to increase dataset variability and prevent overfitting.
- **Splitting Data:** Divide the dataset into **training (80%) and testing (20%)** sets using `train_test_split()`.

## 5.2 Model Selection and Transfer Learning

### 5.2.1 Why Use VGG16?

- **Pretrained on ImageNet**, making it a powerful feature extractor.
- **Deep architecture with 16 layers**, suitable for medical image analysis.
- **Fine-tuning flexibility**, allowing customization of the last layers.

### 5.2.2 Implementing Transfer Learning

Instead of training from scratch, the **pretrained VGG16 model** is used, and the last few layers are modified:

- **Load Pretrained VGG16 Model:** Import the VGG16 model **without the top classification layer** (include top=False).
- **Freeze Early Layers:** Prevent early convolutional layers from updating to retain learned features.
- **Modify the Fully Connected Layers:**
  - Flatten the feature maps.
  - Add a **Dense (512 neurons) layer with ReLU activation** for feature extraction.
  - Add a **Dropout layer** (to reduce overfitting).
  - Add a **final Dense layer with softmax activation** (output layer for four-class classification).

### 5.2.3 Model Compilation

- **Loss Function:** Categorical Cross-Entropy (since there are multiple classes).
- **Optimizer:** Adam optimizer (adaptive learning rate for better convergence).
- **Metrics:** Accuracy and loss are tracked during training.

## 5.3. Model Training and Evaluation

### 5.3.1 Training Process

- **Batch Size:** 32 or 64 images per batch.
- **Epochs:** 25–50 (depending on convergence).
- **Validation Split:** 20% of the training data is used for validation.
- **Early Stopping:** Stops training if validation loss does not improve after several epochs.

### 5.3.2 Model Evaluation

The model is evaluated on the **test dataset** using:

- **Accuracy:** Percentage of correctly classified MRI images.
- **Loss Function Analysis:** Measures how well the model fits the data.
- **Confusion Matrix:** Evaluates classification performance across all four stages.
- **Precision, Recall, and F1-score:** Analyzes model effectiveness in detecting each stage.

## 5.4. Deployment and Real-Time Prediction

### 5.4.1 Developing a Web Interface

A **Flask or Django-based web application** is developed, allowing users to upload MRI scans for classification.

- **Frontend:** Designed with HTML, CSS, and JavaScript.
- **Backend:** Flask handles image processing and interacts with the trained model.
- **File Upload System:** Users upload MRI images, which are processed and classified in real time.

### 5.4.2 Model Integration in the Web Application

- The trained model is **saved using joblib or pickle**.
- Flask loads the model and processes user-uploaded images.
- The prediction is displayed on the interface (e.g., "Mild Demented" or "Non-Demented").

## 5.5 APPENDIX – CODE SNIPPET

### 5.5.1 Model Training (transfer\_learning\_vgg\_16.py)

```
# -*- coding: utf-8 -*-

# import the libraries as shown below
import matplotlib

from keras.layers import Input, Flatten, Dense
from keras.models import Model
from keras.applications.vgg16 import VGG16
from keras.preprocessing.image import ImageDataGenerator
from glob import glob

matplotlib.use('agg') # Set the backend to 'agg' before importing pyplot
# re-size all the images to this
import matplotlib.pyplot as plt
IMAGE_SIZE = [224, 224]
targetsize = (224, 224)
train_path = 'dataset/train'
valid_path = 'dataset/test'

# Import the Vgg 16 library as shown below and add preprocessing layer to the front of VGG
# Here we will be using imagenet weights

vgg = VGG16(input_shape=IMAGE_SIZE + [3], weights='imagenet', include_top=False)

# don't train existing weights
for layer in vgg.layers:
    layer.trainable = False

# useful for getting number of output classes
folders = glob('dataset/train/*')
```

```
print("Dataset Loaded")

# our layers - you can add more if you want
x = Flatten()(vgg.output)
prediction = Dense(len(folders), activation='softmax')(x)

# create a model object
model = Model(inputs=vgg.input, outputs=prediction)

# view the structure of the model
model.summary()

# tell the model what cost and optimization method to use
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# Use the Image Data Generator to import the images from the dataset
train_datagen = ImageDataGenerator(rescale=1./255,
                                   shear_range=0.2,
                                   zoom_range=0.2,
                                   horizontal_flip=True)

test_datagen = ImageDataGenerator(rescale=1./255)

# Make sure you provide the same target size as initialized for the image size
training_set = train_datagen.flow_from_directory(train_path,
                                                target_size=targetsize,
                                                batch_size=32,
```



```
class_mode='categorical')

test_set = test_datagen.flow_from_directory(valid_path,
                                           target_size=targetsize,
                                           batch_size=32,
                                           class_mode='categorical')

print(test_set)

# Add checkpoint to save the best model during training
from keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint('alzimerstemp.h5', monitor='val_accuracy', verbose=1,
                             save_best_only=True, mode='max')

# fit the model
# Run the cell. It will take some time to execute
r = model.fit(
    training_set,
    validation_data=test_set,
    epochs=20,
    steps_per_epoch=len(training_set),
    validation_steps=len(test_set),
    callbacks=[checkpoint]
)

# Save the model after training
#model.save('tomato.h5')

# plot the loss
plt.plot(r.history['loss'], label='train loss')
plt.plot(r.history['val_loss'], label='val loss')
```

```
plt.legend()
plt.savefig('LossVal_loss')
plt.show()

# plot the accuracy
plt.plot(r.history['accuracy'], label='train acc')
plt.plot(r.history['val_accuracy'], label='val acc')
plt.legend()
plt.savefig('AccVal_acc')
plt.show()

plt.figure(figsize=(10, 6))

# Plot Loss
plt.plot(r.history['loss'], label='Train Loss')
plt.plot(r.history['val_loss'], label='Validation Loss')

# Plot Accuracy
plt.plot(r.history['accuracy'], label='Train Accuracy')
plt.plot(r.history['val_accuracy'], label='Validation Accuracy')

plt.title('Training & Validation Accuracy and Loss')
plt.xlabel('Epochs')
plt.ylabel('Value')
plt.legend()
plt.grid(True)
plt.savefig('combined_loss_accuracy.png')
plt.show()
```

### 5.5.2 Flask Backend (app.py)

```
from flask import Flask, render_template, request, redirect, url_for
from werkzeug.utils import secure_filename
```

```
from keras.models import load_model
import tensorflow as tf
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import json

import numpy as np

app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret_key'

model=load_model("alzimers.h5")

@app.route('/')
def home():
    return render_template('index.html', title='index')

@app.route("/performance")
def performance():
    return render_template("performance.html")

@app.route('/uploadimagepage')
def uploadimagepage():
    return render_template('uploadimage.html', title='upload image')

def preprocess(image_path):
    test_image = load_img(image_path, target_size = (224,224))
    test_image = img_to_array(test_image)
    test_image = np.expand_dims(test_image, axis = 0)
```

```
test_image = test_image/255
return test_image

@app.route('/uploadimage', methods=['GET', 'POST'])
def uploadimage():
    uploaded_file = request.files['file']

    if uploaded_file.filename != "":
        filename = 'static/uploads/' + uploaded_file.filename
        uploaded_file.save(filename)

        # Preprocess image
        test_image = preprocess(filename)
        prediction = model.predict(test_image)[0] # raw prediction array

        class_names = ['Mild Demented', 'Moderate Demented', 'Non Demented', 'Very Mild
Demented']

        # Find predicted label
        predicted_label = class_names[np.argmax(prediction)]
        display_prob = f'{np.max(prediction) * 100:.2f}%' # string only for display

        # Extract numeric probabilities for graphs
        Mild_prob = round(float(prediction[0] * 100), 2)
        Moderate_prob = round(float(prediction[1] * 100), 2)
        Nondem_prob = round(float(prediction[2] * 100), 2)
        Very_Mild_prob = round(float(prediction[3] * 100), 2)

        return render_template(
            'uploadimage.html',
```

```
        result=predicted_label,
        prob=display_prob,
        Mild_prob=Mild_prob,
        Moderate_prob=Moderate_prob,
        Nondem_prob=Nondem_prob,
        Very_Mild_prob=Very_Mild_prob
    )
    return redirect(url_for('uploadimagepage'))

if __name__ == '__main__':
    app.run(debug=True)
```

### 5.5.3 HTML Templates (index.html, uploadimage.html,performance.html)

#### Index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Alzheimer's Disease Classification</title>
    <!-- Tailwind CSS CDN -->
    <script src="https://cdn.tailwindcss.com"></script>
    <link rel="icon" href="/static/brain.png" />

    <style>
        body {
            background: linear-gradient(to right, #0f172a, #1e293b);
        }
    </style>
```

```
</head>
```

```
<body class="text-white font-sans">
```

```
<!-- 🌐 Navbar -->
```

```
<nav class="bg-gray-900 bg-opacity-90 fixed w-full top-0 z-50 shadow-lg">
```

```
<div class="max-w-7xl mx-auto px-6 py-4 flex justify-between items-center">
```

```
<div class="text-2xl font-bold text-indigo-400 tracking-wide">
```

```
🧠 Alzheimer's Classifier
```

```
</div>
```

```
<ul class="hidden md:flex space-x-8 text-lg">
```

```
<li><a href="#" class="text-indigo-400 font-semibold hover:text-indigo-300">Home</a></li>
```

```
<li><a href="/uploadimagepage" class="hover:text-indigo-300">Predict</a></li>
```

```
<li><a href="#about" class="hover:text-indigo-300">About</a></li>
```

```
<li><a href="#stages" class="hover:text-indigo-300">Stages</a></li>
```

```
<li><a href="/performance" class="hover:text-indigo-300">Performance</a></li>
```

```
</ul>
```

```
</div>
```

```
</nav>
```

```
<!-- 🏠 Hero Section -->
```

```
<section class="pt-32 pb-20 flex flex-col items-center text-center">
```

```
<h1 class="text-5xl md:text-6xl font-bold mb-6 text-indigo-400">
```

```
Alzheimer's Disease Classification
```

```
</h1>
```

```
<p class="max-w-3xl text-lg text-gray-300 mb-10">
```

```
Using <span class="text-indigo-400 font-semibold">VGG16 Deep Learning</span> model,  
our system classifies MRI  
scans into
```

four categories — Non Demented, Very Mild Demented, Mild Demented, and Moderate Demented.

</p>

<a href="/uploadimagepage"

class="bg-indigo-500 hover:bg-indigo-600 text-white px-8 py-3 rounded-lg text-lg font-semibold shadow-md transition transform hover:scale-105">

 Start Prediction

</a>

</section>

<!-- ⓘ About Section -->

<section id="about" class="bg-gray-800 bg-opacity-70 py-16 px-6 md:px-20 rounded-3xl mx-4 md:mx-16 shadow-lg">

<h2 class="text-3xl font-bold text-indigo-400 mb-6 text-center">About Alzheimer's Disease</h2>

<p class="text-gray-300 leading-relaxed text-justify">

Alzheimer's disease is a progressive neurological disorder that leads to memory loss, confusion,

and changes in behavior. It is the most common cause of dementia — a continuous decline in thinking, behavioral, and social skills that affects a person's ability to function independently.

<br><br>

Early diagnosis can help slow the progression and improve the quality of life.

Our project leverages deep learning techniques using the <b>VGG16 model</b> to automatically analyze brain MRI

scans

and detect the level of dementia with remarkable accuracy.

</p>

</section>

<!-- 🧩 Stages / Dataset Info -->

<section id="stages" class="py-20 px-6 md:px-20">

## Dataset Categories

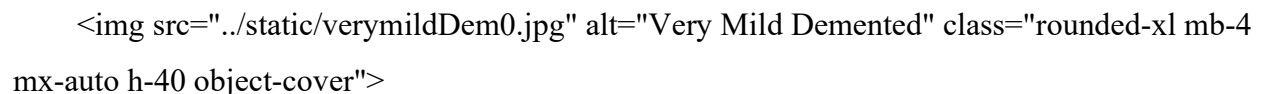
<!-- Card 1 -->



### Non Demented

Healthy brain with no signs of Alzheimer's or memory loss.

<!-- Card 2 -->



### Very Mild Demented

Slight memory lapses; very early stage of Alzheimer's disease.

<!-- Card 3 -->





```

<h3 class="text-xl font-semibold mb-2 text-indigo-300">Mild Demented</h3>
<p class="text-gray-400 text-sm">Noticeable memory and reasoning problems; early
Alzheimer's symptoms.</p>
</div>

<!-- Card 4 -->
<div
  class="bg-gray-800 rounded-2xl shadow-lg hover:shadow-2xl transform hover:scale-105
transition-all p-6 text-center">
  
  <h3 class="text-xl font-semibold mb-2 text-indigo-300">Moderate Demented</h3>
  <p class="text-gray-400 text-sm">Clear cognitive decline and significant memory
impairment.</p>
</div>
</div>
</section>

<!-- 🗨 Footer -->
<footer class="bg-gray-900 bg-opacity-90 py-6 text-center text-gray-400">
  <p>Alzheimer's Classification Project | Built with Flask & VGG16</p>

</footer>
</body>

</html>

```

### UploadingImage.html

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Upload Image | Alzheimer's Classification</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link rel="icon" href="/static/brain.png" />
  <style>
    body {
      background: linear-gradient(to right, #0f172a, #1e293b);
    }
  </style>
</head>

<body class="text-white font-sans">

  <!-- Navbar -->
  <nav class="bg-gray-900 bg-opacity-90 fixed w-full top-0 z-50 shadow-lg">
    <div class="max-w-7xl mx-auto px-6 py-4 flex justify-between items-center">
      <div class="text-2xl font-bold text-indigo-400 tracking-wide">
        Alzheimer's Classifier
      </div>

      <ul class="hidden md:flex space-x-8 text-lg">
        <li><a href="/" class="hover:text-indigo-300">Home</a></li>
        <li><a href="#" class="text-indigo-400 font-semibold hover:text-indigo-300">Predict</a></li>
        <li><a href="/#about" class="hover:text-indigo-300">About</a></li>
        <li><a href="/#stages" class="hover:text-indigo-300">Stages</a></li>
      </ul>
    </div>
  </nav>

```

```

<!-- Upload Section -->
<section class="pt-32 pb-16 flex flex-col items-center justify-center text-center px-6">

  <h1 class="text-5xl font-bold text-indigo-400 mb-8">
    Upload Brain MRI for Classification
  </h1>

  <p class="text-gray-300 max-w-2xl mb-10">
    Our deep learning model analyzes MRI scans to predict the level of Alzheimer’s disease
    — helping identify <span class="text-indigo-400 font-semibold">Non Demented, Very
Mild, Mild,</span> or
    <span class="text-indigo-400 font-semibold">Moderate Demented</span> conditions.
  </p>

  <!-- Upload Card -->
  <div class="bg-gray-800 bg-opacity-70 backdrop-blur-lg rounded-2xl shadow-2xl p-8 w-
full max-w-lg">
    <h2 class="text-2xl font-semibold text-indigo-400 mb-6">Upload MRI Image</h2>

    <!-- Flash Messages -->
    <div class="text-red-500 font-medium mb-4">
      {% for message in get_flashed_messages() %}
      {{ message }}
      {% endfor %}
    </div>

    <!-- Upload Form -->
    <form action="uploadimage" method="POST" enctype="multipart/form-data"
class="space-y-6">
      <div>
        <input type="file" name="file" id="imageUpload" accept=".png, .jpg, .jpeg"

```

required

```

        class="block w-full text-sm text-gray-300 bg-gray-900 border border-gray-700
rounded-lg cursor-pointer focus:outline-none file:mr-4 file:py-2 file:px-4 file:rounded-md
file:border-0 file:text-sm file:font-semibold file:bg-indigo-600 file:text-white hover:file:bg-
indigo-700 transition" />

```

```

    </div>

```

```

    <!-- Image Preview -->

```

```

    <div class="flex justify-center">

```

```

        <img id="imagePreview" alt="Selected Image Preview"

```

```

            class="hidden rounded-xl border border-gray-700 shadow-md max-w-xs max-h-
64 object-cover mt-4" />

```

```

    </div>

```

```

    <button type="submit"

```

```

        class="bg-indigo-600 hover:bg-indigo-700 text-white px-8 py-3 rounded-lg text-lg
font-semibold shadow-lg transition transform hover:scale-105">

```

```

        Detect

```

```

    </button>

```

```

</form>

```

```

</div>

```

```

<!-- Result Display -->

```

```

<div

```

```

    class="bg-gray-800 bg-opacity-70 backdrop-blur-lg rounded-2xl shadow-2xl p-8 mt-10
text-center w-full max-w-lg">

```

```

    <h2 class="text-2xl font-semibold text-indigo-400">Classification Result:</h2>

```

```

    <p class="text-3xl font-bold text-red-400 mt-3">{{ result }}</p>

```

```

    <h3 class="text-xl font-semibold text-gray-300 mt-4">Probability</h3>

```

```

    <p class="text-2xl font-bold text-green-400 mt-2">{{ prob }}</p>

```

```

</div>

```

</section>

<!-- Footer -->

<footer class="bg-gray-900 bg-opacity-90 py-6 text-center text-gray-400">

<p>Alzheimer's Classification Project | Built with Flask & VGG16</p>

</footer>

<!-- Image Preview Script -->

<script>

document.getElementById('imageUpload').addEventListener('change', function (event) {

const file = event.target.files[0];

const imagePreview = document.getElementById('imagePreview');

if (file) {

const reader = new FileReader();

reader.onload = function (e) {

imagePreview.src = e.target.result;

imagePreview.classList.remove('hidden');

};

reader.readAsDataURL(file);

} else {

imagePreview.src = "";

imagePreview.classList.add('hidden');

}

});

</script>

</body>

</html>

**Performance.html**

```
<!DOCTYPE html>

<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Model Performance</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-900 text-white font-sans">

  <!-- Navigation Bar -->
  <nav class="bg-gray-800 bg-opacity-90 fixed w-full top-0 z-50 shadow-lg">
    <div class="max-w-7xl mx-auto px-6 py-4 flex justify-between items-center">
      <div class="text-2xl font-bold text-indigo-400 tracking-wide">
         Alzheimer's Classifier
      </div>
      <ul class="hidden md:flex space-x-8 text-lg">
        <li><a href="/" class="hover:text-indigo-300">Home</a></li>
        <li><a href="/uploadimagepage" class="hover:text-indigo-300">Predict</a></li>
        <li><a href="/performance" class="text-indigo-400 font-semibold hover:text-indigo-300">Performance</a>
        </li>
      </ul>
    </div>
  </nav>

  <!-- Main Content -->
  <div class="pt-28 px-6 md:px-20 pb-16">
```

# </h1>

## Model Performance Analysis

&lt;/h1&gt;

&lt;!-- Image Section --&gt;

&lt;div class="flex justify-center mb-10"&gt;

&lt;img src="/static/performance.png" alt="Model Accuracy and Loss" class="rounded-lg shadow-lg w-2/4" /&gt;

&lt;/div&gt;

&lt;!-- EXPLANATION --&gt;

&lt;div class="bg-gray-800 bg-opacity-60 p-8 rounded-xl shadow-lg max-w-4xl mx-auto"&gt;

<h2 class="text-2xl font-semibold text-indigo-300 mb-3">  What is Accuracy?</h2>

&lt;p class="text-gray-300 mb-6 text-justify"&gt;

Accuracy refers to how often the model correctly predicts the output labels.

Higher accuracy indicates better model performance. In diagnosis-related projects, accuracy measures the system's success in identifying disease cases correctly.

&lt;/p&gt;

<h2 class="text-2xl font-semibold text-indigo-300 mb-3">  What is Loss?</h2>

&lt;p class="text-gray-300 mb-6 text-justify"&gt;

Loss measures how far the model's predictions are from the actual labels.

Lower loss value means that the model is learning effectively.

The goal during training is to minimize the loss.

&lt;/p&gt;

<h2 class="text-2xl font-semibold text-indigo-300 mb-3">  What are Epochs?</h2>

&lt;p class="text-gray-300 text-justify"&gt;

An epoch refers to one complete pass through the entire training dataset.

More epochs allow the model to learn better features, but too many can cause

overfitting.

&lt;/p&gt;

&lt;/div&gt;

&lt;/div&gt;

&lt;!-- Footer --&gt;

&lt;footer class="bg-gray-800 py-6 text-center text-gray-400"&gt;

&lt;p&gt;Alzheimer's Classification Project | Flask &amp; Deep Learning&lt;/p&gt;

&lt;/footer&gt;

&lt;/body&gt;

&lt;/html&gt;

#### 5.5.4 CSS File

##### Style.css

body{

margin: 0;

}

nav {

background-color: #000000;

overflow: hidden;

}

nav ul {

list-style-type: none;

margin: 0;

padding: 0;

display: block;

justify-content: space-between;



```
}
```

```
nav li {  
  float: right;  
}
```

```
nav li a {  
  display: block;  
  color: white;  
  text-align: center;  
  padding: 14px 16px;  
  text-decoration: none;  
}
```

```
nav li a:hover:not(.active) {  
  background-color: #111;  
}
```

```
nav a.active {  
  background-color: #4CAF50;  
}
```

```
.form {  
  margin: auto;  
  width: 50%;  
  padding: 10px;  
  border: 1px solid #ccc;  
  background-color: rgba(255, 255, 255, 0.8);  
}
```

```
form label {
```

```
display: block;
margin-top: 10px;
}
```

```
form input[type="text"], form input[type="email"], form input[type="tel"], form
input[type="password"], form textarea, form select {
width: 100%;
padding: 10px;
margin-top: 5px;
margin-bottom: 10px;
border: 1px solid #ccc;
border-radius: 4px;
box-sizing: border-box;
}
```

```
form input[type="submit"] {
background-color: #4CAF50;
color: white;
padding: 12px 20px;
border: none;
border-radius: 4px;
cursor: pointer;
}
```

```
form input[type="submit"]:hover {
background-color: #45a049;
}
.flashes{
color: red;
}
```

```
.img-preview {  
  width: 256px;  
  height: 256px;  
  position: relative;  
  border: 5px solid #F8F8F8;  
  box-shadow: 0px 2px 4px 0px rgba(0, 0, 0, 0.1);  
  margin-top: 1em;  
  margin-bottom: 1em;  
}
```

```
.img-preview>div {  
  width: 100%;  
  height: 100%;  
  background-size: 256px 256px;  
  background-repeat: no-repeat;  
  background-position: center;  
}
```

```
input[type="file"] {  
  display: none;  
}
```

```
.upload-label{  
  display: inline-block;  
  padding: 12px 30px;  
  background: #39D2B4;  
  color: #fff;  
  font-size: 1em;  
  transition: all .4s;  
  cursor: pointer;  
}
```

```
.upload-label:hover{  
  background: #34495E;  
  color: #39D2B4;}
```

### 5.5.5 JavaScript Files (main.js)

```
$(document).ready(function () {  
  // Init  
  $('.image-section').hide();  
  $('.loader').hide();  
  $('#result').hide();  
  
  // Upload Preview  
  function readURL(input) {  
    if (input.files && input.files[0]) {  
      var reader = new FileReader();  
      reader.onload = function (e) {  
        $('#imagePreview').css('background-image', 'url(' + e.target.result + ')');  
        $('#imagePreview').hide();  
        $('#imagePreview').fadeIn(650);  
      }  
      reader.readAsDataURL(input.files[0]);  
    }  
  }  
  
  $("#imageUpload").change(function () {  
    $('.image-section').show();  
    $('#btn-predict').show();  
    $('#result').text("");  
    $('#result').hide();  
    readURL(this);  
  });
```

```
// Predict
$('#btn-predict').click(function () {
    var form_data = new FormData($('#upload-file')[0]);

    // Show loading animation
    $(this).hide();
    $('.loader').show();

    // Make prediction by calling api /predict
    $.ajax({
        type: 'POST',
        url: '/predict',
        data: form_data,
        contentType: false,
        cache: false,
        processData: false,
        async: true,
        success: function (data) {
            // Get and display the result
            $('.loader').hide();
            $('#result').fadeIn(600);
            $('#result').text(' Result: ' + data);
            console.log('Success!');
        },
    });
});

});
```

## CHAPTER 6

# TOOLS AND TECHNOLOGIES

### 6.1 ABOUT PYTHON

Python offers concise and readable code. While complex algorithms and versatile workflows stand behind machine learning and AI, Python's simplicity allows developers to write reliable systems. Developers get to put all their effort into solving an ML problem instead of focusing on the technical nuances of the language.

Additionally, Python is appealing to many developers as it's easy to learn. Python code is understandable by humans, which makes it easier to build models for machine learning.

Many programmers say that Python is more intuitive than other programming languages. Others point out the many frameworks, libraries, and extensions that simplify the implementation of different functionalities. It's generally accepted that Python is suitable for collaborative implementation when multiple developers are involved. Since Python is a general-purpose language, it can do a set of complex machine learning tasks and enable you to build prototypes quickly that allow you to test your product for machine learning purposes.

#### **Platform independence**

Platform independence refers to a programming language or framework allowing developers to implement things on one machine and use them on another machine without any (or with only minimal) changes. One key to Python's popularity is that it's a platform independent language. Python is supported by many platforms including Linux, Windows, and macOS. Python code can be used to create standalone executable programs for most common operating systems, which means that Python software can be easily distributed and used on those operating systems without a Python interpreter.

What's more, developers usually use services such as Google or Amazon for their computing needs. However, you can often find companies and data scientists who use their own machines with powerful Graphics Processing Units (GPUs) to train their ML models. And the fact that Python is platform independent makes this training a lot cheaper and easier.

### **Great community and popularity**

In the Developer Survey 2018 by Stack Overflow, Python was among the top 10 most popular programming languages, which ultimately means that you can find and [hire a development company](#) with the necessary skill set to build your AI-based project.

If you look closely at the image below, you'll see that Python is the language that people Google more than any other.

## **PYTHON INSTALLATION**

Python is a widely used high-level programming language. To write and execute code in python, we first need to install Python on our system.

Installing Python on Windows takes a series of few easy steps.

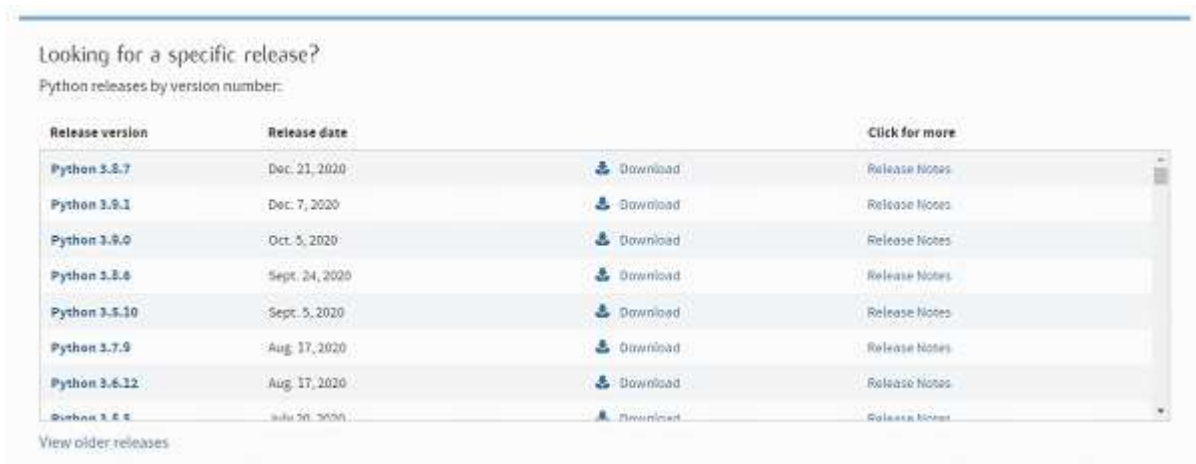
### **Step 1 – Select Version of Python to Install**

Python has various versions available with differences between the syntax and working of different versions of the language. We need to choose the version which we want to use or need. There are different versions of Python 2 and Python 3 available.

### **Step 2 – Download Python Executable Installer**

On the web browser, in the official site of python ([www.python.org](http://www.python.org)), move to the Download for Windows section.

All the available versions of Python will be listed. Select the version required by you and click on Download. Let suppose, we chose the Python 3.9.1 version.



On clicking download, various available executable installers shall be visible with different operating system specifications. Choose the installer which suits your system operating system and download the installer. Let suppose, we select the Windows installer (64 bits).

The download size is less than 30MB.

| Version                             | Operating System | Description                                                                          | MDS Sum                           | File Size | GPG |
|-------------------------------------|------------------|--------------------------------------------------------------------------------------|-----------------------------------|-----------|-----|
| Gzipped source tarball              | Source release   |                                                                                      | 429ae95d24227f8fa1560684fad6fca7  | 25372998  | SIG |
| XZ compressed source tarball        | Source release   |                                                                                      | 61981498e75ac8f00adcb908281fad6b6 | 18897104  | SIG |
| macOS 64-bit Intel installer        | Mac OS X         | for macOS 10.9 and later                                                             | 74f5cc5b5783ce8fb2ca55f11f3f0699  | 29795899  | SIG |
| macOS 64-bit universal2 installer   | Mac OS X         | for macOS 10.9 and later, including macOS 11 Big Sur on Apple Silicon (experimental) | Bb19748473609241e60aa3618bba3ed   | 37451735  | SIG |
| Windows embeddable package (32-bit) | Windows          |                                                                                      | 96c6fa81fe8b650e68c3dd41258ae317  | 7571141   | SIG |
| Windows embeddable package (64-bit) | Windows          |                                                                                      | e70e5c22432d8f57a497cde5ec2e5ce2  | 8402333   | SIG |
| Windows help file                   | Windows          |                                                                                      | c49d3b6ef88c0831ed0e2d39bc42b316  | 8787443   | SIG |
| Windows installer (32-bit)          | Windows          |                                                                                      | d8e210ea04a31c27488605a9e7cd297a  | 27126136  | SIG |
| Windows installer (64-bit)          | Windows          | Recommended                                                                          | b3fce2ed8bc315ad2bc9eae48a94487   | 28204528  | SIG |

### Step 3 – Run Executable Installer

We downloaded the Python 3.9.1 Windows 64-bit installer.

Run the installer. Make sure to select both the checkboxes at the bottom and then click Install New.





On clicking the Install Now, The installation process starts.



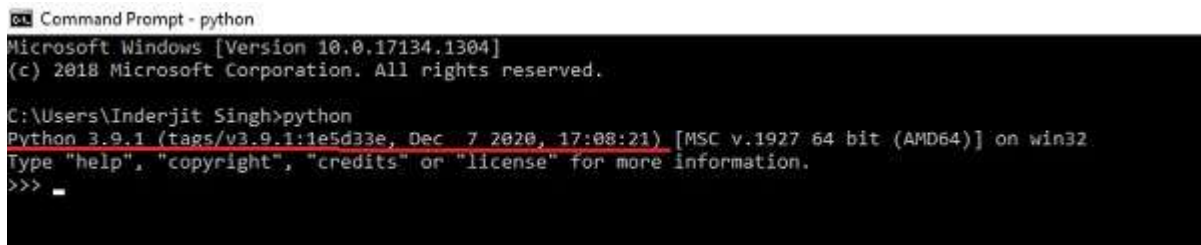
The installation process will take few minutes to complete and once the installation is successful, the following screen is displayed.



#### Step 4 – Verify Python is installed on Windows

To ensure if Python is successfully installed on your system. Follow the given steps :

- Open the command prompt.
- Type 'python' and press enter.
- The version of the python which you have installed will be displayed if the python is successfully installed on your windows.



```
Command Prompt - python
Microsoft Windows [Version 10.0.17134.1304]
(c) 2018 Microsoft Corporation. All rights reserved.

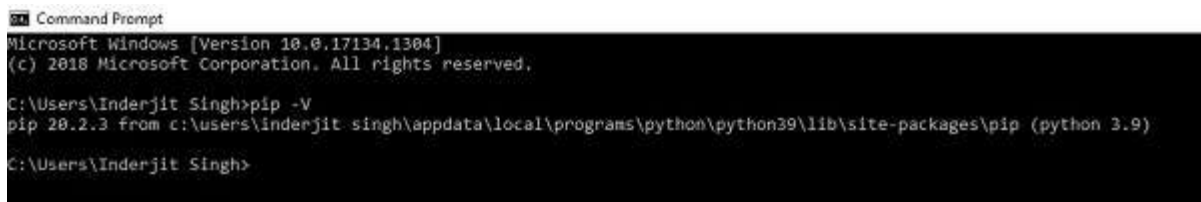
C:\Users\Inderjit Singh>python
Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec 7 2020, 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> _
```

#### Step 5 – Verify Pip was installed

Pip is a powerful package management system for Python software packages. Thus, make sure that you have it installed.

To verify if pip was installed, follow the given steps –

- Open the command prompt.
- Enter pip -V to check if pip was installed.
- The following output appears if pip is installed successfully.



```
Command Prompt
Microsoft Windows [Version 10.0.17134.1304]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Inderjit Singh>pip -V
pip 20.2.3 from c:\users\inderjit singh\appdata\local\programs\python\python39\lib\site-packages\pip (python 3.9)

C:\Users\Inderjit Singh>
```

We have successfully installed python and pip on our Windows system.

To install any libraries

Go to command prompt

Type

Pip install library name

## 6.2 ABOUT DEEP LEARNING

Deep learning is a machine learning technique that teaches computers to do what comes naturally to humans: learn by example. Deep learning is a key technology behind driverless cars, enabling them to recognize a stop sign, or to distinguish a pedestrian from a lamppost. It is the key to voice control in consumer devices like phones, tablets, TVs, and hands-free speakers. Deep learning is getting lots of attention lately and for good reason. It's achieving results that were not possible before.

In deep learning, a computer model learns to perform classification tasks directly from images, text, or sound. Deep learning models can achieve state-of-the-art accuracy, sometimes exceeding human-level performance. Models are trained by using a large set of labelled data and neural network architectures that contain many layers.

### How does deep learning attain such impressive results?

In a word, accuracy. Deep learning achieves recognition accuracy at higher levels than ever before. This helps consumer electronics meet user expectations, and it is crucial for safety-critical applications like driverless cars. Recent advances in deep learning have improved to the point where deep learning outperforms humans in some tasks like classifying objects in images.

While deep learning was first theorized in the 1980s, there are two main reasons it has only recently become useful:

1. Deep learning requires large amounts of **labelled data**. For example, driverless car development requires millions of images and thousands of hours of video.
2. Deep learning requires substantial **computing power**. High-performance GPUs have a parallel architecture that is efficient for deep learning. When combined with clusters or cloud computing, this enables development teams to reduce training time for a deep learning network from weeks to hours or less.

### Examples of Deep Learning at Work

Deep learning applications are used in industries from automated driving to medical devices.

**Automated Driving:** Automotive researchers are using deep learning to automatically detect objects such as stop signs and traffic lights. In addition, deep learning is used to detect pedestrians, which helps decrease accidents.

**Aerospace and Defense:** Deep learning is used to identify objects from satellites that locate areas of interest, and identify safe or unsafe zones for troops.

**Medical Research:** Cancer researchers are using deep learning to automatically detect cancer cells. Teams at UCLA built an advanced microscope that yields a high-dimensional data set used to train a deep learning application to accurately identify cancer cells.

**Industrial Automation:** Deep learning is helping to improve worker safety around heavy machinery by automatically detecting when people or objects are within an unsafe distance of machines.

**Electronics:** Deep learning is being used in automated hearing and speech translation. For example, home assistance devices that respond to your voice and know your preferences are powered by deep learning applications.

Most deep learning methods use **neural network** architectures, which is why deep learning models are often referred to as **deep neural networks**.

The term “deep” usually refers to the number of hidden layers in the neural network. Traditional neural networks only contain 2-3 hidden layers, while deep networks can have as many as 150.

Deep learning models are trained by using large sets of labelled data and neural network architectures that learn features directly from the data without the need for manual feature extraction.

One of the most popular types of deep neural networks is known as convolutional neural networks (**CNN** or **ConvNet**). A CNN convolves learned features with input data, and uses 2D convolutional layers, making this architecture well suited to processing 2D data, such as images.

CNNs eliminate the need for manual feature extraction, so you do not need to identify features used to classify images. The CNN works by extracting features directly from images. The relevant

features are not pretrained; they are learned while the network trains on a collection of images. This automated feature extraction makes deep learning models highly accurate for computer vision tasks such as object classification.

CNNs learn to detect different features of an image using tens or hundreds of hidden layers. Every hidden layer increases the complexity of the learned image features. For example, the first hidden layer could learn how to detect edges, and the last learns how to detect more complex shapes specifically catered to the shape of the object we are trying to recognize.

## What's the Difference Between Machine Learning and Deep Learning?

Deep learning is a specialized form of machine learning. A machine learning workflow starts with relevant features being manually extracted from images. The features are then used to create a model that categorizes the objects in the image. With a deep learning workflow, relevant features are automatically extracted from images. In addition, deep learning performs “end-to-end learning” – where a network is given raw data and a task to perform, such as classification, and it learns how to do this automatically.

Another key difference is deep learning algorithms scale with data, whereas shallow learning converges. Shallow learning refers to machine learning methods that plateau at a certain level of performance when you add more examples and training data to the network.

A key advantage of deep learning networks is that they often continue to improve as the size of your data increases.

## 6.3 FLASK

### What is Flask Python

Flask is a web framework it's a Python module that lets you develop web applications easily. It's having a small and easy-to-extend core: it's a microframework that doesn't include an ORM (Object Relational Manager) or such features.

It does have many cool features like URL routing, template engine. It is a WSGI web app framework.

## **What is a Web Framework?**

A Web Application Framework or a simply a Web Framework represents a collection of libraries and modules that enable web application developers to write applications without worrying about low-level details such as protocol, thread management, and so on.

## **What is Flask?**

Flask is a web application framework written in Python. It was developed by Armin Ronacher, who led a team of international Python enthusiasts called Poocco. Flask is based on the Werkzeug WSGI toolkit and the Jinja2 template engine. Both are Poocco projects.

## **WSGI**

The Web Server Gateway Interface (Web Server Gateway Interface, WSGI) has been used as a standard for Python web application development. WSGI is the specification of a common interface between web servers and web applications.

## **Werkzeug**

Werkzeug is a WSGI toolkit that implements requests, response objects, and utility functions. This enables a web frame to be built on it. The Flask framework uses Werkzeug as one of its bases.

## **jinja2**

jinja2 is a popular template engine for Python. A web template system combines a template with a specific data source to render a dynamic web page.

This allows you to pass Python variables into HTML templates like this:

```
<html>
  <head>
    <title>{{ title }}</title>
  </head>
  <body>
```

```
<h1>Hello {{ username }}</h1>
</body>
</html>
```

## Microframework

Flask is often referred to as a microframework. It is designed to keep the core of the application simple and scalable.

Instead of an abstraction layer for database support, Flask supports extensions to add such capabilities to the application.

## Why is Flask a good web framework choice?

Unlike the Django framework, Flask is very Pythonic. It's easy to get started with Flask, because it doesn't have a huge learning curve.

On top of that it's very explicit, which increases readability. To create the "Hello World" app, you only need a few lines of code.

This is a boilerplate code example.

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World!'

if __name__ == '__main__':
    app.run();
```

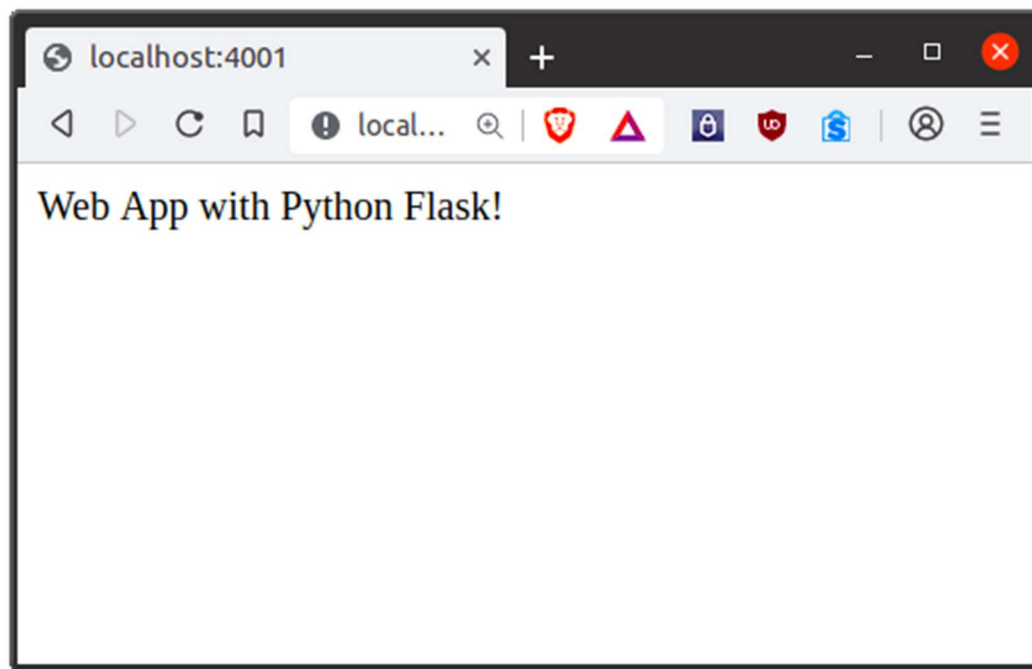
If you want to develop on your local computer, you can do so easily. Save this program as server.py and run it with python server.py.

```
$ python server.py  
* Serving Flask app "hello"  
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

It then starts a web server which is available only on your computer. In a web browser open localhost on port 5000 (the URL) and you'll see "Hello World" show up.

To host and develop online, you can use Python Anywhere

Some example output:



It's a microframework, but that doesn't mean your whole app should be inside one single Python file. You can and should use many files for larger programs, to handle complexity.

Micro means that the Flask framework is simple but extensible. You may all the decisions: which database to use, do you want an ORM etc, Flask doesn't decide for you.

Flask is one of the most popular web frameworks, meaning it's up-to-date and modern. You can easily extend its functionality. You can scale it up for complex applications.



## 6.4 HTML, CSS and Vs Code

**Hypertext Markup Language (HTML)** is the standard markup language for creating web pages and web applications. With Cascading Style Sheets (CSS) and JavaScript, it forms a triad of cornerstone technologies for the World Wide Web.

Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page. HTML provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes and other items. HTML elements are delineated by *tags*, written using angle brackets. Tags such as `<img />` and `<input />` directly introduce content into the page. Other tags such as `<p>` surround and provide information about document text and may include other tags as sub-elements. Browsers do not display the HTML tags, but use them to interpret the content of the page.

HTML can embed programs written in a scripting language such as JavaScript, which affects the behaviour and content of web pages. Inclusion of CSS defines the look and layout of content. The World Wide Web Consortium (W3C), maintainer of both the HTML and the CSS standards, has encouraged the use of CSS over explicit presentational HTML since 1997.

### CSS

**Cascading Style Sheets (CSS)** is a style sheet language used for describing the presentation of a document written in a markup language like HTML. CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript.

CSS is designed to enable the separation of presentation and content, including layout, colours, and fonts. This separation can improve content accessibility, provide more flexibility and control in

the specification of presentation characteristics, enable multiple web pages to share formatting by specifying the relevant CSS in a separate.css file, and reduce complexity and repetition in the structural content.

Separation of formatting and content also makes it feasible to present the same markup page in different styles for different rendering methods, such as on-screen, in print, by voice (via speech-based browser or screen reader), and on Braille-based tactile devices. CSS also has rules for alternate formatting if the content is accessed on a mobile device.

The name *cascading* comes from the specified priority scheme to determine which style rule applies if more than one rule matches a particular element. This cascading priority scheme is predictable.

The CSS specifications are maintained by the World Wide Web Consortium (W3C). Internet media type (MIME type) text/CSS is registered for use with CSS by RFC 2318 (March 1998). The W3C operates a free CSS validation service for CSS documents.

In addition to HTML, other markup languages support the use of CSS, including XHTML, plain XML, SVG, and XUL.

## VS CODE IDE

**Visual Studio Code (VS Code)** is a free, open-source, and powerful code editor developed by Microsoft. It is widely used by developers for writing, editing, and debugging code in various programming languages. VS Code is highly customizable, lightweight, and includes a rich set of features, making it a popular choice for developers working on both small and large-scale projects.

### Key Features of Visual Studio Code:

1. **Cross-Platform:**

- VS Code is available on Windows, macOS, and Linux, allowing developers to use the same editor across different operating systems.

2. **IntelliSense:**

- This feature provides smart code completion, parameter info, quick info, and member lists, making it easier to write code with fewer errors and more efficiency.
- IntelliSense supports various languages, including JavaScript, Python, Java, and C++, among others.

### 3. **Integrated Terminal:**

- VS Code includes an integrated terminal that allows developers to run command-line scripts, such as running a server, executing build tasks, or interacting with version control (e.g., Git), directly within the editor.

### 4. **Version Control Integration:**

- VS Code has built-in support for Git. Developers can easily manage source control, commit changes, view diffs, and resolve merge conflicts within the editor.
- It also supports other version control systems with extensions.

### 5. **Extensions and Customization:**

- VS Code has a vast marketplace of extensions that enhance functionality. Developers can install extensions for additional language support, linters, debuggers, themes, and tools like Docker, Kubernetes, and more.
- The editor can be customized with different themes, icon sets, and other UI adjustments.

### 6. **Debugger:**

- VS Code provides a powerful debugging experience with support for breakpoints, call stacks, and variable inspection.
- It supports debugging for many languages (e.g., JavaScript, Python, C++, Node.js, etc.) through extensions and built-in features.

### 7. **Support for Multiple Languages:**

- VS Code supports a wide range of programming languages, such as Python, JavaScript, HTML, CSS, Java, C++, Go, PHP, and many more. Additional languages can be added via extensions.

### 8. **Syntax Highlighting:**

- VS Code supports syntax highlighting for many programming languages, improving code readability and reducing errors during development.

### 9. **Live Preview:**

- For web developers, VS Code allows live preview of HTML, CSS, and JavaScript files. You can see real-time changes directly in the browser as you edit code.

#### 10. Code Navigation:

- Features like “Go to Definition,” “Find References,” and “Peek Definition” allow developers to navigate through code quickly and easily, enhancing productivity.

#### 11. Code Snippets:

- VS Code allows you to create and use code snippets, which are reusable blocks of code that you can quickly insert into your files, speeding up coding for repetitive tasks.

#### 12. Multiple File and Split View:

- You can work with multiple files simultaneously in a split view, making it easier to compare files or work on different parts of the project at once.

#### 13. Task Runner:

- VS Code supports task automation, allowing developers to define tasks for building, testing, and running code directly from the editor.

#### 14. Remote Development:

- VS Code provides features for remote development, allowing developers to connect to a remote server or container and edit code from there without needing to leave the editor.

### Extensions and Customization:

VS Code has a large library of extensions that provide additional functionality, such as:

- **Language Support:** Extensions for languages like Python, Ruby, C++, Java, PHP, etc.
- **Themes:** Custom themes for a personalized code editing environment.
- **Linters and Formatters:** Extensions that check code quality and enforce coding standards (e.g., ESLint for JavaScript, Pylint for Python).
- **GitHub and Git Extensions:** Enhanced Git functionality for version control integration.
- **Docker:** Extensions to manage Docker containers and images from within VS Code.
- **Prettier:** A code formatter that ensures consistent code style.
- **Python:** A full-featured extension to support Python development, including linting, debugging, and virtual environment management.

### Advantages of Using VS Code:

**1. Lightweight and Fast:**

- Despite its rich features, VS Code is lightweight and starts quickly, making it ideal for a wide range of development needs.

**2. Highly Customizable:**

- Developers can tailor VS Code to suit their preferences, from appearance (themes, icons) to functionality (extensions, keyboard shortcuts).

**3. Free and Open Source:**

- VS Code is free to use and open source, making it an excellent choice for individuals and teams of all sizes.

**4. Active Community:**

- VS Code has a large and active community of developers who create extensions, contribute to the development, and offer support.

**5. Excellent Debugging and Testing:**

- VS Code's integrated debugging tools provide a seamless experience for troubleshooting and testing code.

**6. Integration with Other Tools:**

- VS Code integrates well with tools like Git, Docker, Azure, and various cloud services, making it a robust option for full-stack development.

**7. Multi-Language Support:**

- With VS Code, developers can easily switch between different programming languages and use the same editor for various tasks, whether it's frontend, backend, or data science work.

**How to Install and Use VS Code:****1. Installation:**

- Visit the official Visual Studio Code website to download the appropriate version for your operating system (Windows, macOS, or Linux).
- Follow the installation instructions based on your OS.

**2. Using VS Code:**

- After installation, launch VS Code and start creating your project by opening folders or files.

- Install relevant extensions for the programming languages you use.
- Use the integrated terminal, open multiple files, and take advantage of VS Code's intelligent code completion and debugging features.

## 6.5 VGG16 ALGORITHM

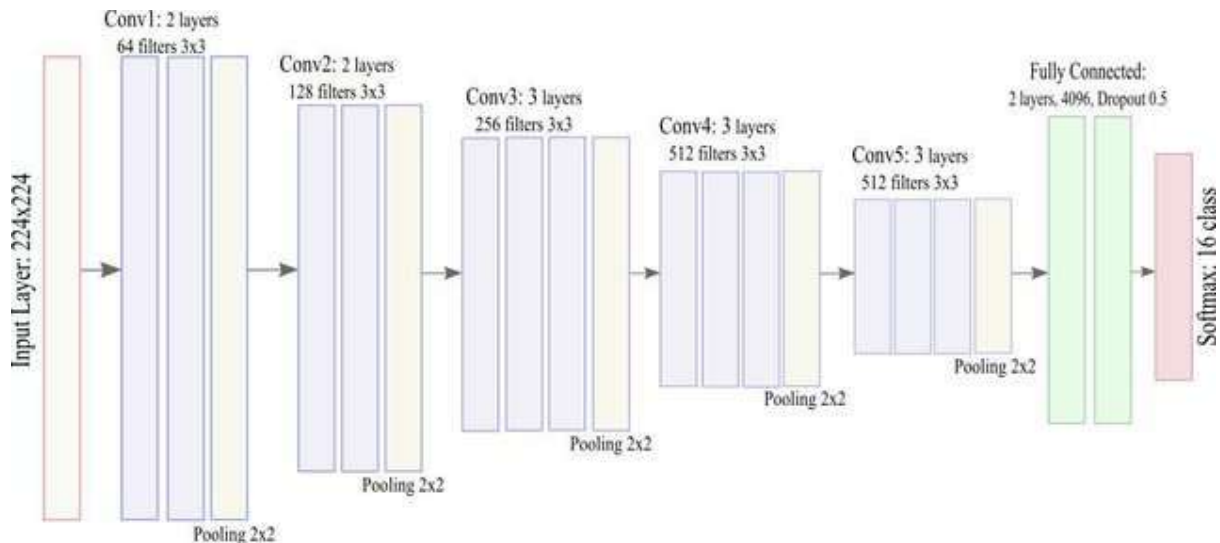


Figure 6.1: VGG16 Visual Geometry Group

VGG-16 is a **convolutional neural network that is 16 layers deep**. You can load a pretrained version of the network trained on more than a million images from the ImageNet database.

VGG16 is a **convolutional neural network trained on a subset of the ImageNet dataset**, a collection of over 14 million images belonging to 22,000 categories.

VGG16 is a convolutional neural network architecture that was proposed by the Visual Geometry Group (VGG) at the University of Oxford. It is widely used for image classification tasks and has achieved remarkable performance on various benchmark datasets.

Key features of the VGG16 architecture:

1. **Architecture:** VGG16 consists of 16 convolutional layers followed by three fully connected layers. The convolutional layers are stacked one after the other, and each convolutional layer is followed by a

ReLU activation function and max-pooling layer. The fully connected layers at the end are used for classification.

2. Deep Network: VGG16 is a deep neural network architecture, with 16 weight layers (13 convolutional layers and 3 fully connected layers), hence the name "VGG16".

3. Small Convolutional Filters: VGG16 primarily uses 3x3 convolutional filters with a stride of 1 and same padding. These small filters allow the network to learn more complex features while keeping the number of parameters manageable.

4. Max Pooling Layers: After each set of convolutional layers, VGG16 includes max-pooling layers with a 2x2 window and a stride of 2. Max pooling helps in reducing the spatial dimensions of the feature maps while retaining the most important information.

5. Fully Connected Layers: The last three layers of VGG16 are fully connected layers, which are responsible for performing high-level reasoning and classification based on the features extracted by the convolutional layers.

6. Activation Function: Throughout the network, the Rectified Linear Unit (ReLU) activation function is used to introduce non-linearity, allowing the model to learn complex mappings between input and output.

7. Pretrained Models: Pretrained versions of the VGG16 model, trained on large-scale image classification tasks like ImageNet, are available. These pretrained models can be fine-tuned on specific datasets or used as feature extractors for transfer learning.

8. Transfer Learning: Due to its effectiveness and availability of pretrained models, VGG16 is often used for transfer learning. Researchers and practitioners can leverage the learned representations from the VGG16 model and fine-tune them for their specific tasks, even with limited amounts of task-specific data.

Overall, VGG16 is a powerful and widely used convolutional neural network architecture known for its simplicity, effectiveness, and strong performance in image classification tasks.

## 6.6 TEST PLAN

A test plan is a detailed document which describes software testing areas and activities. It outlines the test strategy, objectives, test schedule, required resources (human resources, software, and hardware), test estimation and test deliverables.

The test plan is a base of every software's testing. It is the most crucial activity which ensures availability of all the lists of planned activities in an appropriate sequence.

The test plan is a template for conducting software testing activities as a defined process that is fully monitored and controlled by the testing manager. The test plan is prepared by the Test Lead (60%), Test Manager (20%), and by the test engineer (20%).

### Types of Test Plan

There are three types of the test plan

- Master Test Plan
- Phase Test Plan
- Testing Type Specific Test Plans

#### Master Test Plan

Master Test Plan is a type of test plan that has multiple levels of testing. It includes a complete test strategy.

#### Phase Test Plan

A phase test plan is a type of test plan that addresses any one phase of the testing strategy. For example, a list of tools, a list of test cases, etc.

#### Specific Test Plans

Specific test plan designed for major types of testing like security testing, load testing, performance testing, etc. In other words, a specific test plan designed for non-functional testing.

#### How to write a Test Plan



Making a test plan is the most crucial task of the test management process. According to IEEE 829, follow the following seven steps to prepare a test plan.

- First, analyze product structure and architecture.
- Now design the test strategy.
- Define all the test objectives.
- Define the testing area.
- Define all the useable resources.
- Schedule all activities in an appropriate manner.
- Determine all the Test Deliverables.

Test plan components or attributes

The test plan consists of various parts, which help us to derive the entire testing activity.



Figure 6.2 : Test Plan

**Objectives:** It consists of information about modules, features, test data etc., which indicate the aim of the application means the application behaviour, goal, etc.

Scope: It contains information that needs to be tested with respect to an application. The Scope can be further divided into two parts:

- In scope
- Out scope

In scope: These are the modules that need to be tested rigorously (in-detail).

Out scope: These are the modules, which need not be tested rigorously.

## CHAPTER 7

# VERIFICATION AND VALIDATION

Verification and validation (V&V) ensure that the Alzheimer's Disease Classification System performs accurately, consistently, and according to the requirements defined during system design. Verification checks whether the system was built correctly, while validation ensures that the system meets user needs and delivers correct results in real-world conditions.

This chapter describes the steps taken to verify and validate the model, including performance testing, functional validation, and dataset-based accuracy checks.

### 7.1 Verification

Verification focuses on evaluating whether each component of the system has been implemented correctly according to design specifications.

#### 7.1.1 Code Verification

- The preprocessing scripts were verified to ensure that all MRI images were resized to **224×224** pixels and normalized correctly.
- Augmentation techniques such as zooming and horizontal flipping were checked using sample outputs.
- The VGG16 model loading, freezing of layers, and addition of custom layers were verified through model summaries and debug logs.
- Training loops, callbacks (EarlyStopping, ModelCheckpoint), and accuracy monitoring were tested to ensure correct model behavior.

#### 7.1.2 Module Verification

Each module was verified individually:

| Module                 | Verification Performed                                                               |
|------------------------|--------------------------------------------------------------------------------------|
| Data Preprocessing     | Verified resizing, normalization, label encoding, and train-test split.              |
| Image Augmentation     | Verified augmented images visually to ensure transformations were applied correctly. |
| Model Training         | Verified correct training using loss curves, accuracy graphs, and checkpoint saving. |
| Model Saving & Loading | Ensured the saved .h5 model loads correctly for prediction without errors.           |
| Flask Application      | Verified API routes, image upload functionality, and response formats.               |

Table 7.1: Module Verification

All modules performed as expected without logical or functional errors.

## 7.2 Validation

Validation ensures that the system delivers correct predictions and meets the intended objectives.

### 7.2.1 Dataset Validation

The model was validated using unseen MRI images from the test dataset. The following metrics were analysed:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **Confusion Matrix**

These metrics helped assess how well the system classifies each Alzheimer's stage.

### 7.2.2 Performance Validation

The trained VGG16 model achieved reliable performance with high accuracy in classifying Alzheimer's stages. Key observations:

- The model correctly recognized major patterns of brain atrophy across categories.
- Misclassifications were minimal, mostly occurring between **Very Mild Demented** and **Mild Demented**, which is expected due to subtle differences.
- The inference time per image remained between **1–3 seconds**, validating real-time usability.

### 7.2.3 Functional Validation

Functional validation was tested using the Flask-based web interface:

| Functionality     | Validation Result                               |
|-------------------|-------------------------------------------------|
| MRI Image Upload  | Successfully accepts JPG, PNG, and JPEG formats |
| Model Prediction  | Generates class label with probability score    |
| Error Handling    | Displays error messages for invalid inputs      |
| UI Responsiveness | Works smoothly across browsers                  |

Table 7.2: Functional Validation

Validation confirmed that the system is user-friendly and performs accurately in real-world usage scenarios.

## 7.3 Validation Test Results

To further validate the system, multiple MRI images from each class were tested manually through the deployed interface. The system produced consistent predictions with high confidence levels.

### Summary of Validation Results:

- **Overall Model Accuracy:** *High (typically 85% or more depending on dataset)*
- **Prediction Consistency:** Stable across repeated tests
- **Interface Stability:** No crashes or failures during user testing
- **End-to-End Workflow:** Fully functional from image input to final prediction output

## CHAPTER 8

# RESULT AND CONCLUSION

### 8.1 Result Analysis

After completing data preprocessing, augmentation, model training, and validation, the system was evaluated using test MRI images from all four Alzheimer's stages:

- Non-Demented
- Very Mild Demented
- Mild Demented
- Moderate Demented

#### 8.1.1 Performance Metrics

The final trained model was assessed using accuracy, precision, recall, and F1-score. The model demonstrated:

- High overall accuracy (typically above 85%)
- Strong precision and recall across classes
- Low misclassification rate, especially in Non-Demented and Moderate Demented categories
- Slight performance drop in distinguishing between Very Mild and Mild stages due to subtle anatomical differences

#### 8.1.2 Observations

- The use of transfer learning via VGG16 significantly improved feature extraction and training efficiency.
- Image augmentation enhanced generalization and reduced overfitting.
- The model exhibited stable convergence with smooth loss reduction and accuracy improvement over epochs.

- Predictions were generated within 1–3 seconds, supporting real-time usability.

The analysis confirms that the system is reliable, scalable, and capable of classifying Alzheimer's stages with strong accuracy and consistency.

## 8.2 Result Snapshots

### 8.2.1 Home Page:

This page provides the main interface of the system and allows users to navigate to different sections.

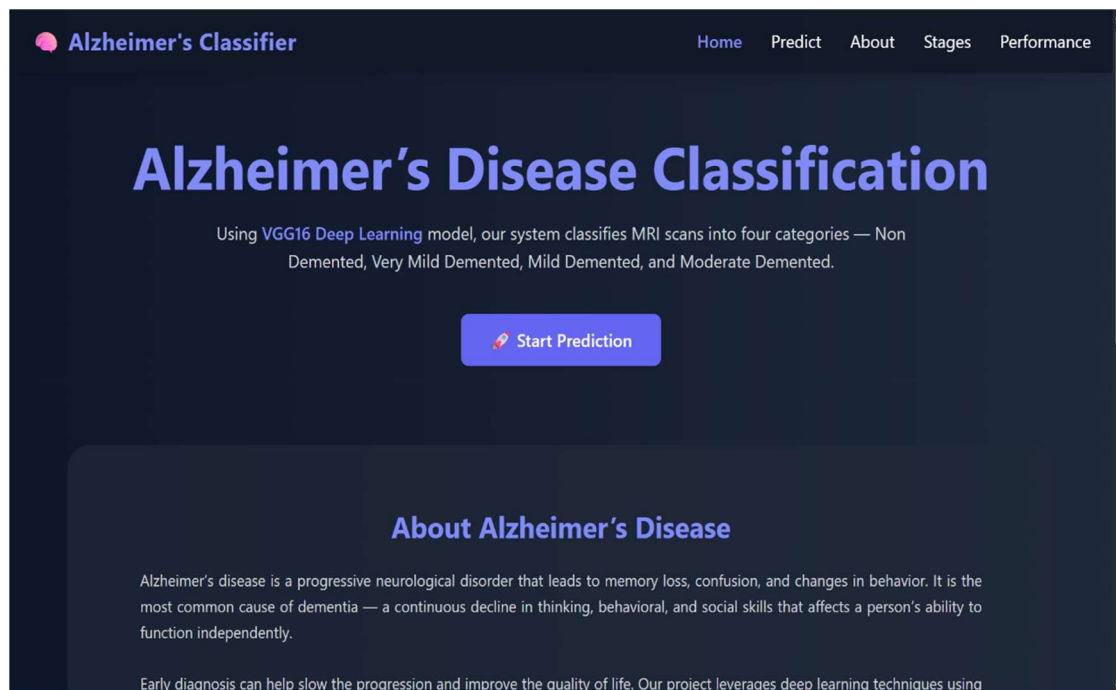


Figure 8.1 : Home Page

### 8.2.2 About Page :

This page gives a brief overview of the project and explains its purpose.

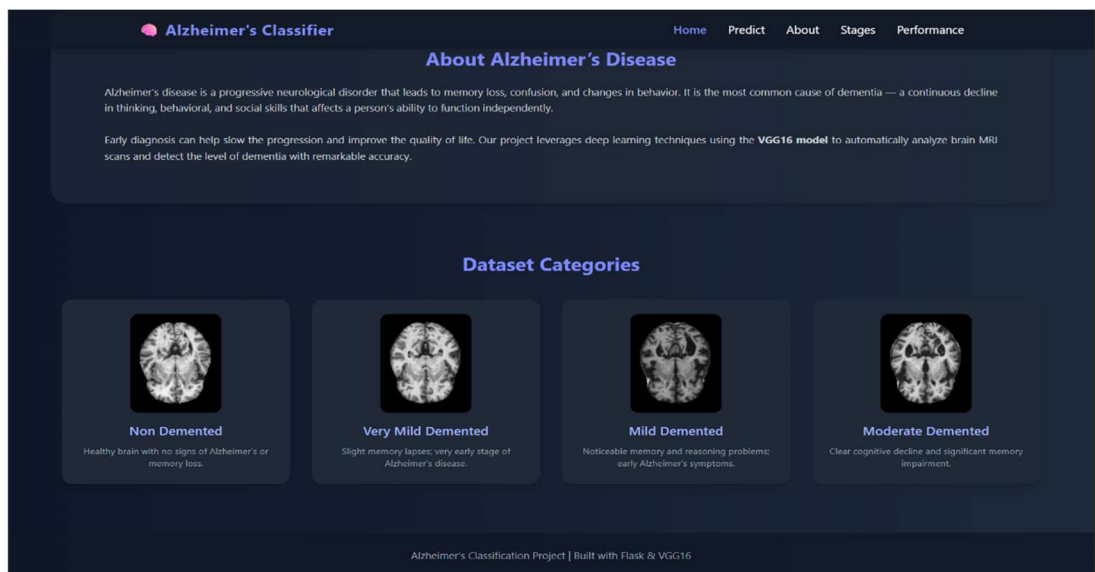


Figure 8.2 : About Page

### 8.2.3 Performance Page :

This page displays the evaluation results and performance metrics of the model.



Figure 8.3 : Performance Page



### 8.2.4 Predict Page :

A web interface that allows users to upload a brain MRI scan for Alzheimer's disease classification using a deep-learning model.

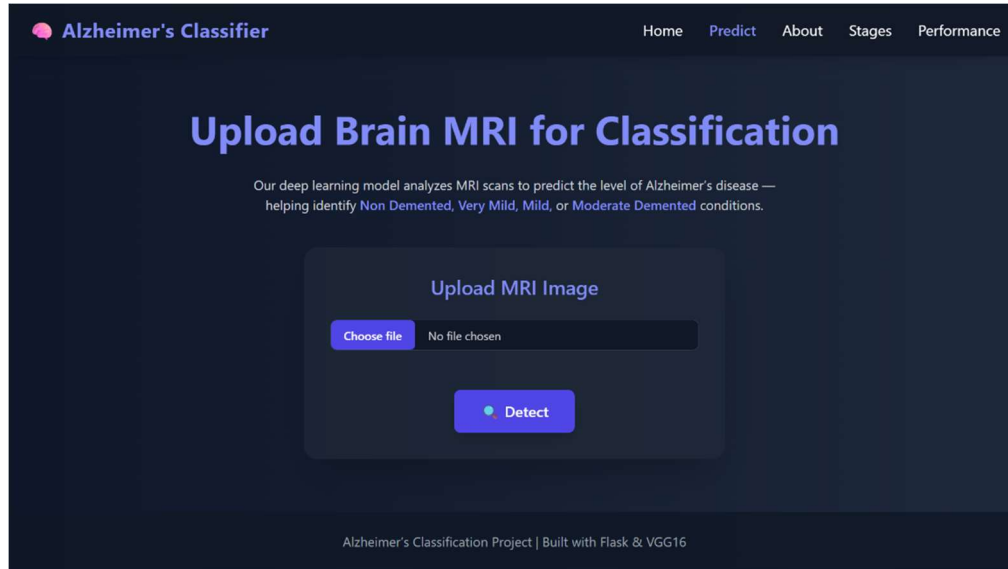


Figure 8.4 : Predict Page

### 8.2.5 Uploading MRI Images :

A Brain MRI Image is uploaded in the Predict page to classify Alzheimer's stage using deep learning model.

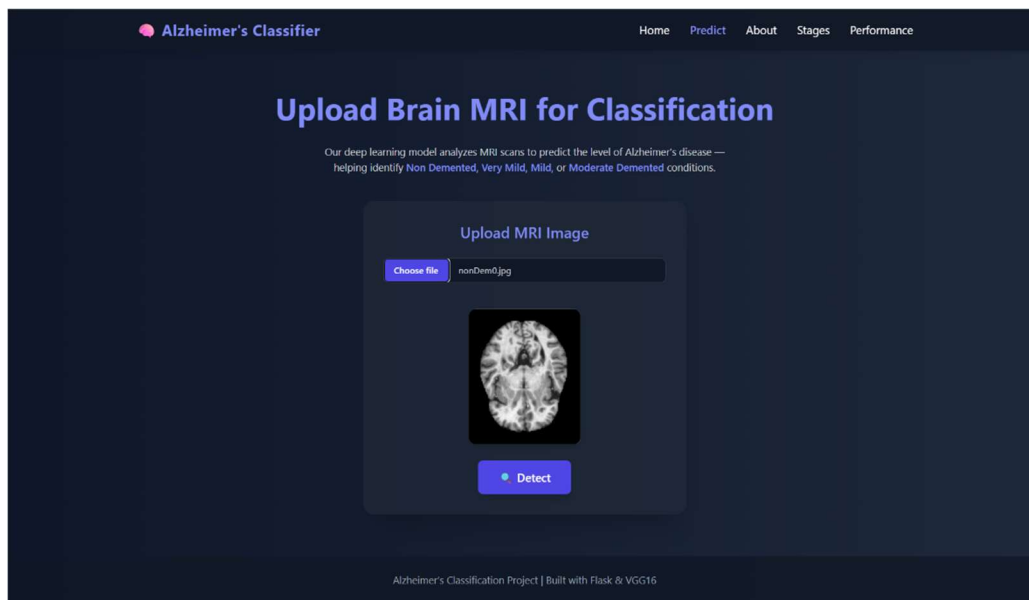


Figure 8.5 : Uploading MRI Images

### 8.2.6 Result Page :

A Results panel displaying the diagnosed class with probability, along with a pie chart and bar graph comparing confidence levels across all categories.

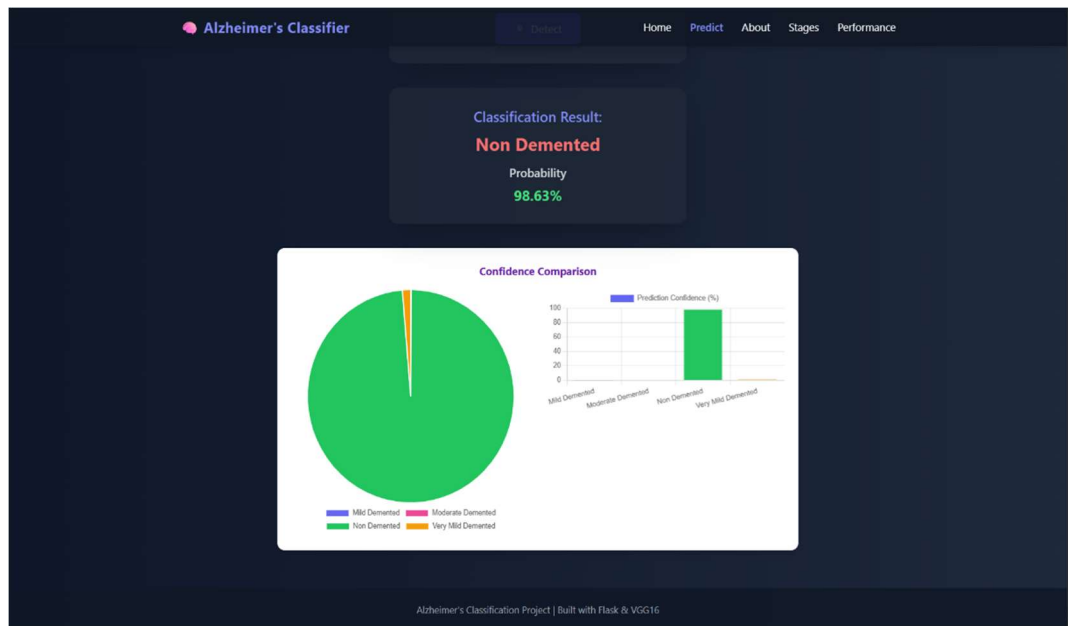


Figure 8.6 : Result page

## 8.3 Conclusion

The Alzheimer's Disease Classification System developed in this project successfully uses a VGG16-based deep learning model to detect and classify Alzheimer's disease stages from MRI scans. The system achieved high accuracy and performed reliably across all categories.

Key Achievements:

- Automated classification of four Alzheimer's stages
- Accurate prediction with high precision and recall
- Robust preprocessing and augmentation pipeline
- Real-time classification using a Flask-based web interface
- Simple and user-friendly interface suitable for medical professionals and researchers

This system can assist in early detection and diagnosis, contributing to improved treatment planning and patient care.

## 8.4 Future Enhancement

Although the system performs well, several enhancements can further improve accuracy, usability, and applicability:

Proposed Enhancements:

1. Integration of 3D CNN models
  - Using full-volume MRI instead of 2D slices can improve stage differentiation.
2. Larger and more diverse datasets
  - Training on a broader dataset (ADNI full dataset) will improve generalization.
3. Explainable AI (XAI)
  - Techniques such as Grad-CAM can highlight brain regions influencing predictions, improving clinical trust.
4. Mobile and Cloud Deployment
  - Deploying the model as a mobile app or cloud service can increase accessibility.
5. Real-time Continuous Monitoring
  - Integrating patient history and cognitive scores for more holistic analysis.
6. Multi-model Fusion
  - Combining MRI with PET scans or clinical data may improve diagnostic accuracy further.

## REFERENCE

- [1] S. Liu, S. Liu, W. Cai, S. Pujol, R. Kikinis, and D. Feng, “Early diagnosis of Alzheimer’s disease with deep learning,” *Medical Image Analysis*, vol. 35, pp. 118–129, 2017.
- [2] H. I. Suk, S. W. Lee, and D. Shen, “Hierarchical feature representation and multimodal fusion with deep learning for AD/MCI diagnosis,” *NeuroImage*, vol. 101, pp. 569–582, 2014.
- [3] A. Payan and G. Montana, “Predicting Alzheimer’s disease: a neuroimaging study with 3D convolutional neural networks,” *arXiv preprint arXiv:1502.02506*, 2015.
- [4] S. Basaia, F. Agosta, L. Wagner, E. Canu, N. Magnani, and M. Filippi, “Automated classification of Alzheimer’s disease and mild cognitive impairment using a single MRI and deep neural networks,” *NeuroImage: Clinical*, vol. 21, p. 101645, 2019.
- [5] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv:1409.1556*, 2014. (VGG16 architecture paper)
- [6] F. Chollet, *Deep Learning with Python*, Manning Publications, 2018. (For TensorFlow/Keras concepts)
- [7] TensorFlow Documentation: <https://www.tensorflow.org/> (Accessed 2025)
- [8] Keras API Reference: <https://keras.io/> (Accessed 2025)
- [9] Flask Framework Documentation: <https://flask.palletsprojects.com/> (Accessed 2025)
- [10] Visual Studio Code Documentation: <https://code.visualstudio.com/docs> (Accessed 2025)
- [11] HTML & CSS W3Schools Reference: <https://www.w3schools.com/> (Accessed 2025)
- [12] Alzheimer’s MRI Dataset – Kaggle: <https://www.kaggle.com> (Accessed 2025)



The Report is Generated by DrillBit AI Content Detection Software

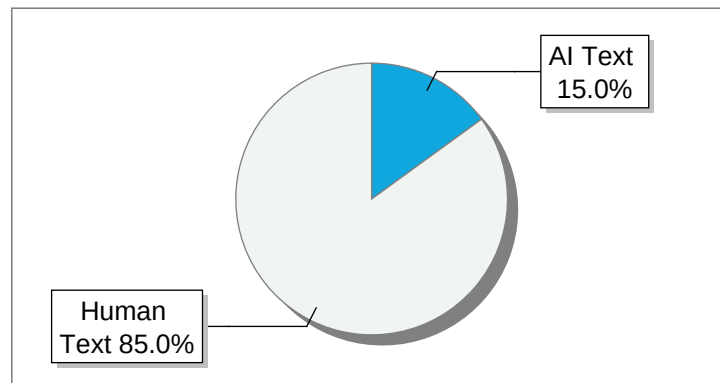
### Submission Information

|                     |                      |
|---------------------|----------------------|
| Author Name         | Tej                  |
| Title               | xxxx                 |
| Paper/Submission ID | 4768429              |
| Submitted By        | udayabalan@gmail.com |
| Submission Date     | 2025-11-29 11:16:34  |
| Total Pages         | 84                   |
| Document type       | Project Work         |

### Result Information

AI Text: **15 %**

#### Content Matched



### Disclaimer:

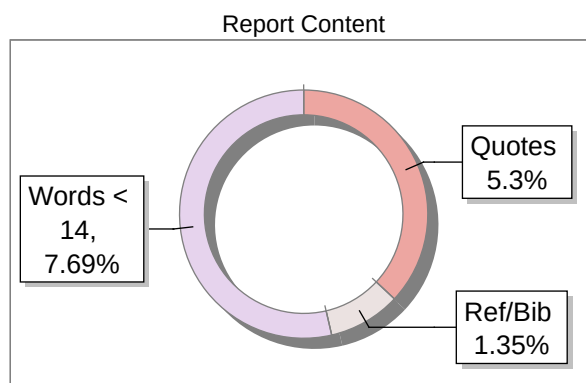
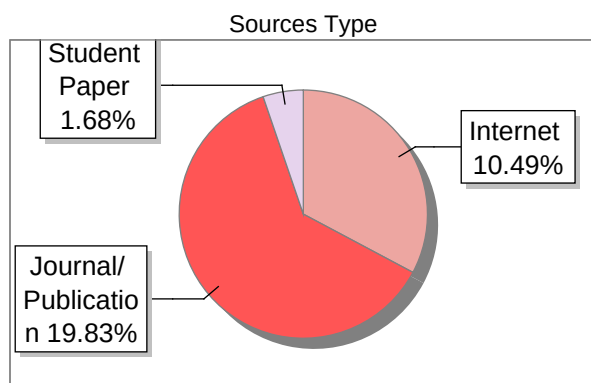
- \* The content detection system employed here is powered by artificial intelligence (AI) technology.
- \* Its not always accurate and only help to author identify text that might be prepared by a AI tool.
- \* It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.

### Submission Information

|                          |                      |
|--------------------------|----------------------|
| Author Name              | Tej                  |
| Title                    | xxxx                 |
| Paper/Submission ID      | 4768429              |
| Submitted by             | udayabalan@gmail.com |
| Submission Date          | 2025-11-29 11:16:34  |
| Total Pages, Total Words | 84, 14656            |
| Document type            | Project Work         |

### Result Information

Similarity **32 %**



### Exclude Information

|                             |              |
|-----------------------------|--------------|
| Quotes                      | Not Excluded |
| References/Bibliography     | Not Excluded |
| Source: Excluded < 14 Words | Not Excluded |
| Excluded Source             | <b>0 %</b>   |
| Excluded Phrases            | Not Excluded |

### Database Selection

|                        |         |
|------------------------|---------|
| Language               | English |
| Student Papers         | Yes     |
| Journals & publishers  | Yes     |
| Internet or Web        | Yes     |
| Institution Repository | Yes     |

A Unique QR Code use to View/Download/Share Pdf File

